

VELLORE INSTITUTE OF TECHNOLOGY

Theory of Computation — Project Report

Student Attendance System

using DFA-based Registration Validation and FSM-based Attendance Tracking

with Error Detection and Recovery

Submitted by

Name	Register Number
B. Harish	24BIT0154
B. Abhishek	24BIT0323

Programme: B.Tech — Information Technology

Course: Theory of Computation

October 2026

GitHub repository: https://github.com/HARISH0789/TOC-Student-Attendance-System

Contents

1. Aim of the Project 4
2. Project Title 4
3. Abstract 4
4. Register Numbers 5
5. Students' Names (Team Members) 5
6. Tools Used 6
7. Highlights / Features of the Project 6
7.1 Novelty — what makes this project different 6
7.2 Theory of Computation concepts demonstrated 7
8. Project Modules 8
9. Module Description 9
9.1 Module 1 — Registration Number Validation (DFA) 9
9.2 Module 2 — Student Database & Data Storage 11
9.3 Module 3 — Attendance Management (FSM) 11
9.4 Module 4 — Graphical User Interface 12
9.5 Module 5 — Timer & Control 13
9.6 Module 6 — Attendance Analytics (75 % rule) 14
9.7 Module 7 — Report Export 15
10. Role of Team Members 16
11. Software Description 16
11.1 Overview 16
11.2 Requirements 16
11.3 Installation and running 16
11.4 Project structure 16
11.5 Workflow 17
11.6 Testing 17
12. Coding 18
12.1 Module 1 — dfa.py 18
12.2 Module 3 — fsm.py 23
12.3 Module 5 — session.py 24
12.4 Module 2 — storage.py 27
12.5 Module 6 — analytics.py 31
12.6 Module 4 — GUI: key event handlers (attendance/gui/app.py) 32

12.7 main.py 34
13. Screenshots of the Error Segments 35
14. Screenshots of the Error Recovered Segments 42
15. Possible Future Work 47
16. Conclusion 47
17. References 48

1. Aim of the Project

To design and implement a Student Attendance System that applies Theory of Computation concepts to a real classroom problem:

- a **Deterministic Finite Automaton (DFA)** that accepts exactly the regular language of valid registration numbers, explains where and why an invalid number is rejected, and suggests a safe correction;
- a **Finite State Machine (FSM)**, driven by a session timer, that tracks every student's attendance from NOT_MARKED to PRESENT, LATE, ABSENT, EXCUSED or ON_DUTY and rejects illegal events such as double marking;
- reliable **error detection and recovery** for invalid input, duplicate records, missing sessions, time-outs, corrupted data files and application crashes.

The result is a complete, tested desktop application in which the theory (DFA, FSM, timer) is kept separate from the user interface.

2. Project Title

Student Attendance System using DFA-based Registration Validation and FSM-based Attendance Tracking, with Error Detection and Recovery

Short title: Student Attendance System — DFA + FSM.

3. Abstract

Manual attendance is slow and error-prone. Mistyped registration numbers create bad records, a student can be marked twice or by proxy, late arrivals are judged inconsistently, and a crashed spreadsheet can lose a whole class's data. This project treats these problems formally.

The set of valid VIT registration numbers is a regular language, so it is recognised by a DFA M = (Q, Σ, δ, q0, F). The DFA is compiled from simple patterns such as `##@@@####` (# = digit, @ = letter). Several department formats are combined by building an NFA for their union, applying the subset construction and then Moore's minimisation; three formats give 24 states, which reduce to 13. Every keystroke is a DFA transition. The interface therefore shows the live state while typing, rejects an input at the exact position where the DFA enters the dead state, and offers a DFA-guided correction (for example `24bit 03S7` → `24BIT0357`).

Each student's attendance is a deterministic FSM with six states and six events. A session timer turns a mark into MARK_ON_TIME or MARK_LATE and fires TIMEOUT when the window closes. Undefined transitions, such as a second mark, are rejected and logged, which prevents duplicate and proxy marking. The system is written in Python with Tkinter. It adds an animated DFA and FSM visualizer, crash recovery for interrupted sessions, quarantine of corrupted CSV rows, 75 % attendance analytics and Excel export. 67 automated tests check the automata and every recovery path.

Keywords: deterministic finite automaton, regular language, subset construction, DFA minimisation, finite state machine, error recovery, attendance system.

4. Register Numbers

- **24BIT0154**
- **24BIT0323**

5. Students' Names (Team Members)

S. No.	Name	Register Number	Programme
1	B. Harish	24BIT0154	B.Tech — Information Technology
2	B. Abhishek	24BIT0323	B.Tech — Information Technology

6. Tools Used

Tool / Library	Version	Used for
Python	3.13 (runs on 3.10+)	Programming language for the whole system
Tkinter / ttk (standard library)	Tk 8.6	Desktop GUI; Canvas used to draw and animate the DFA and FSM diagrams
csv, json, pathlib, tempfile (standard library)	—	CSV database, crash-recovery snapshot, atomic file writes
openpyxl	3.1	Excel (.xlsx) report export (CSV fallback if missing)
pytest	9.x	Automated testing — 67 test cases
Pillow (ImageGrab)	11.x	Tool that captures the report screenshots from the real app
matplotlib	3.10	System architecture figure
Git & GitHub	2.46	Version control and project tracking (module-wise commits)
Visual Studio Code	—	Code editor / IDE
Windows 11	—	Development and demo platform

7. Highlights / Features of the Project

- **Live DFA validation while typing.** The entry box and a mini state diagram show the current state (q0…q9), the dead state qd, or acceptance after every key press.
- **Explained rejections.** For example: “Rejected at position 3: symbol 'b' is not allowed in state q2 (expected a letter A-Z)”, together with the full DFA path.
- **DFA-guided “Did you mean …?” recovery.** The input is cleaned (separators removed, uppercase) and look-alikes are swapped (O→0, S→5, 1→I), but only where the DFA allows the swap.
- **Any ID format without code changes.** Patterns are compiled into a DFA. Several patterns form a union that is built with the **subset construction** and reduced with **Moore's minimisation**.
- **Strict attendance FSM.** Six states, six events and a separate start state. Undefined δ(state, event) is rejected and logged, which stops double and proxy marking.
- **Timer & control.** The on-time window gives PRESENT, the late window gives LATE, and at the end of the window TIMEOUT automatically moves every unmarked student to ABSENT.
- **New ON_DUTY state** (VIT OD) and **RESET (undo)** for faculty mistakes during a session.
- **Crash recovery.** An interrupted session is restored on the next start, with no marks lost.
- **Corrupted-data recovery.** Bad CSV rows are quarantined to a backup file and the good rows still load. All writes are atomic.
- **Automata visualizers.** An animated DFA run on an input tape (Run / Step), and an FSM diagram that highlights the selected student's state and last transition.
- **75 % rule analytics.** Attendance % for each student, red rows below 75 %, and the number of classes in a row needed to recover. Excel export with CSV fallback.
- **Audit trail.** Every rejected input and every FSM transition is logged.

7.1 Novelty — what makes this project different

Area	A typical attendance system	This project
ID check	One fixed format, checked with if/else or a regex	DFA compiled from patterns; several formats combined with the subset construction and Moore minimisation;

Area	A typical attendance system	This project
		formal definition and δ-table generated automatically
Invalid ID	A generic “invalid” message	Position, state, expected symbol class and DFA path; DFA-guided auto-correction
While typing	No feedback until Submit	Live state (live / dead / final) with colour and a mini state diagram
Attendance status	A status value written to a file; a student can be marked twice	One strict FSM per student (6 states, 6 events); undefined δ is rejected and logged; transition history
Timing	Fixed cut-off or decided by hand	ON_TIME / LATE phases and automatic TIMEOUT
Reliability	Errors hidden, or the app stops	Crash snapshot + restore, quarantine of corrupted rows, atomic writes
Visualisation	—	Animated DFA (tape + head, Step/Run) and FSM state diagram
Reports	A raw attendance list	75 % analytics with classes needed, Excel/CSV export, reject & audit logs
Testing	Manual testing	67 automated pytest tests + scripted GUI screenshots

7.2 Theory of Computation concepts demonstrated

- Alphabet, strings and formal languages: Σ = {0-9, A-Z}, L(M) = set of valid IDs.
- DFA as a 5-tuple, the extended transition function δ*, final, live and dead (trap) states.
- NFA and conversion to a DFA with the subset construction (Rabin–Scott).
- Closure of regular languages under union.
- DFA minimisation with Moore's partition refinement (state equivalence).
- Finite state machines as event-driven systems: $\delta : S \times E \rightarrow S$, a deterministic partial function.
- Error recovery: phrase-level correction, as in compiler design.

8. Project Modules

No.	Module	Source file	Concept / responsibility
1	Registration Validation (DFA)	attendance/dfa.py	Pattern compiler, subset construction, minimisation, trace, suggestions
2	Student Database & Storage	attendance/storage.py	CSV files, duplicate check, quarantine, atomic writes, snapshot, logs
3	Attendance Management (FSM)	attendance/fsm.py	States, events, transition function δ, history
4	Graphical User Interface	attendance/gui/	Five-tab Tkinter app, live DFA, visualizers
5	Timer & Control	attendance/session.py	ON_TIME / LATE / TIMEOUT, crash snapshot / restore
6	Attendance Analytics	attendance/analytics.py	75 % rule, classes needed
7	Report Export	attendance/exporter.py	Excel workbook, CSV fallback

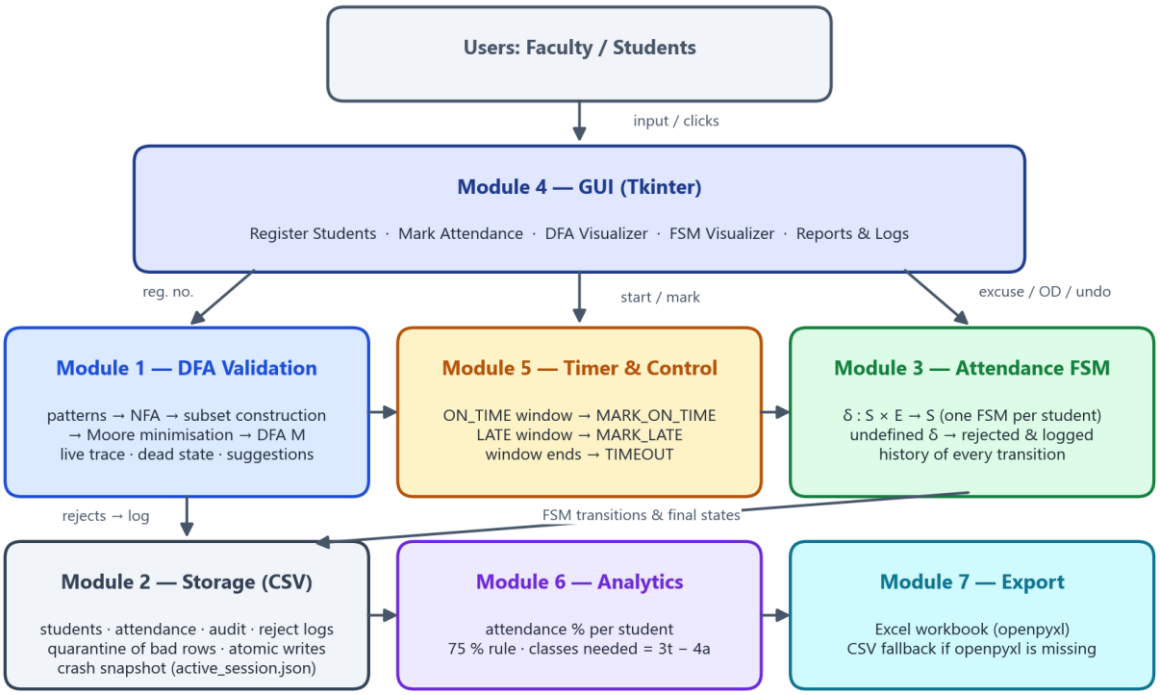


Figure 1: *System architecture: how the seven modules work together.*

9. Module Description

9.1 Module 1 — Registration Number Validation (DFA)

A registration number is accepted only if it belongs to the regular language L of valid IDs. VIT numbers have the form **year (2 digits) + programme code (3 letters) + roll number (4 digits)**, e.g. 24BIT0154. The default pattern is therefore ##@@@####.

Formal definition of the default DFA

```
M  = (Q, Σ, δ, q0, F)
Q  = {q0, q1, q2, q3, q4, q5, q6, q7, q8, q9, qd}      (qd = dead / trap state)
Σ  = {0,1,...,9, A,B,...,Z}                              (36 symbols)
q0 = start state
F  = {q9}
δ  = table below; every missing entry goes to qd, and δ(qd, a) = qd for all a ∈ Σ
L(M) = { d d l l l d d d d  |  d ∈ 0-9, l ∈ A-Z }
```

State	on digit 0-9	on letter A-Z	Type
q0	q1	qd	start
q1	q2	qd	
q2	qd	q3	
q3	qd	q4	
q4	qd	q5	
q5	q6	qd	
q6	q7	qd	
q7	q8	qd	
q8	q9	qd	
q9	qd	qd	final (accept)
qd	qd	qd	dead (trap)

Pattern compiler

The DFA is not hand-coded; it is generated from patterns:

Symbol in pattern	Meaning	Example
#	any digit 0-9	## = year
@	any letter A-Z	@@@ = programme
A-Z, 0-9	that exact symbol	24BIT#### = only the 2024 IT batch
,	separates patterns (union)	24BIT####, 24BDE####, 23BIT####

1. **Parse** every pattern into a chain of allowed-symbol sets (one per position). Each chain is a small NFA whose states are (pattern, position).
2. **Union + subset construction:** a DFA state is a set of NFA states. Start = {(p, 0) for every pattern p}. δ(S, a) = {(p, i+1) | (p, i) ∈ S and a is allowed at position i of p}. The empty set is the dead state qd. A set is final if it contains the end of some pattern.
3. **Moore minimisation:** start with the partition {F, Q − F} (qd included) and split blocks until no symbol can tell two states of a block apart. Equivalent states are merged. For 24BIT####, 24BDE####, 23BIT#### the subset construction gives **24 states** and minimisation reduces them to **13**.

4. States are renamed q0, q1, … in breadth-first order, and the formal definition and δ-table are generated for display.

Run, live states and explained rejection

`trace()` runs M symbol by symbol and records every step δ(q, a) = q'. Example runs:

```
24BIT0154 : q0 -2-> q1 -4-> q2 -B-> q3 -I-> q4 -T-> q5 -0-> q6 -1-> q7 -5-> q8 -4-> q9   ACCEPT (q9 ∈ F)
24B1T0154 : q0 -2-> q1 -4-> q2 -B-> q3 -1-> qd                                    REJECT
            "Rejected at position 4: symbol '1' is not allowed in state q3 (expected a letter (A-Z))"
24BIT99   : ... ends in q7                                                      REJECT
            "input ended in state q7, which is not final. 2 more symbol(s) needed"
```

While typing, a non-empty prefix is either in a **live** state (a final state can still be reached: amber), in the **dead** state qd (red), or in a **final** state (green). Runs take **O(n)** time, with one table lookup per symbol.

Error recovery: DFA-guided correction

1. Remove separators (space - _ / .) and convert to uppercase.
2. Simulate M. When δ(q, a) = qd, try a look-alike symbol (O→0, I→1, S→5, Z→2, B→8 … or 0→O, 1→I, 5→S …).
3. Accept the swap **only if δ(q, swap) ≠ qd**, so the correction is guided by the automaton. If no safe swap exists, no suggestion is made.
4. Suggest the result only if the corrected string is accepted (ends in F).

Examples: `24bit 03S7` → `24BIT0357`, `24B1T0154` → `24BIT0154`, `24BIT0O54` → `24BIT0054`.

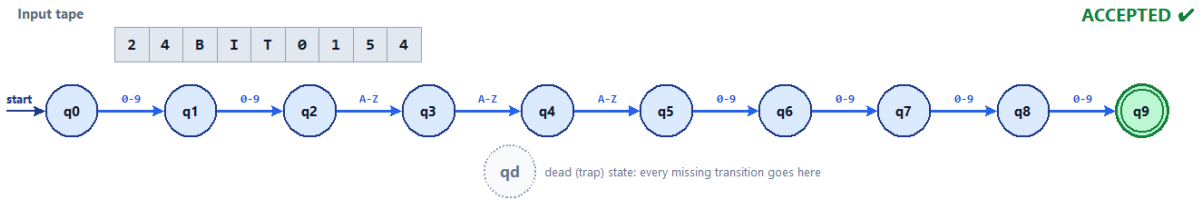


Figure 2: *DFA visualizer: the state diagram generated from ##@@@#### and the accepting run of 24BIT0154 on the input tape.*

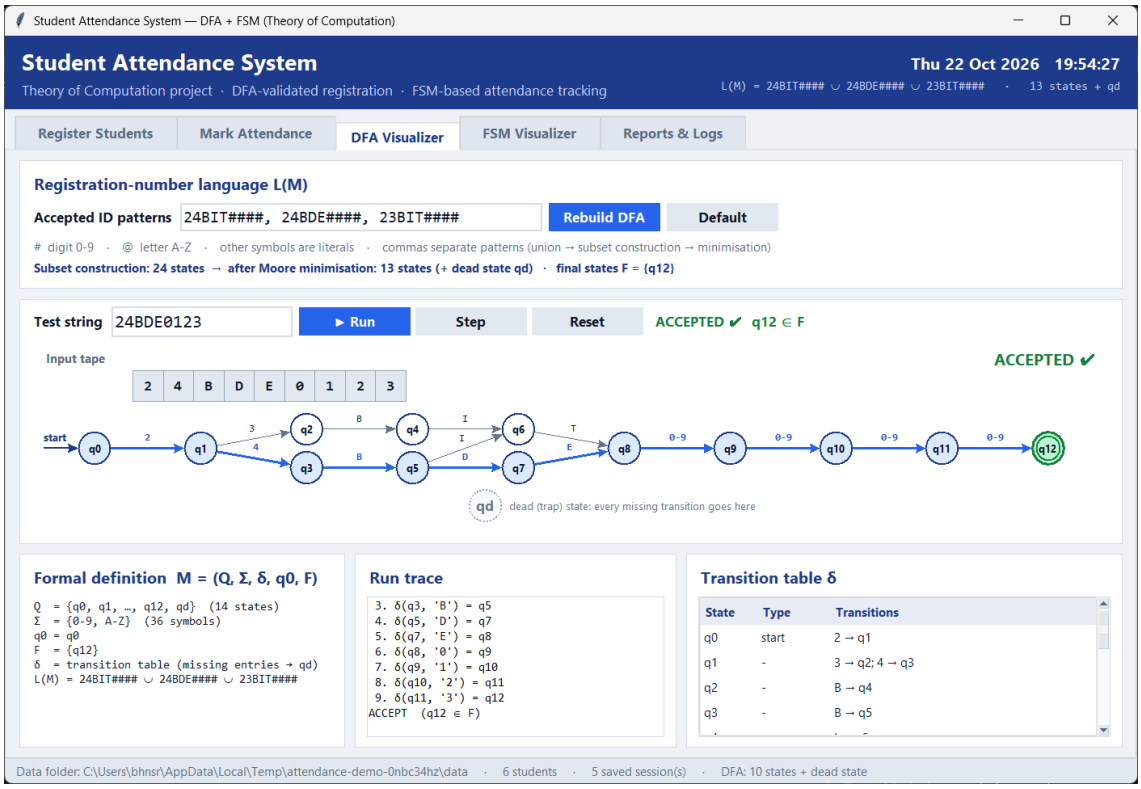


Figure 3: *Three ID formats: the union DFA built by subset construction (24 states) and minimised to 13 states.*

9.2 Module 2 — Student Database & Data Storage

File	Contents
students.csv	RegNo, Name, DateAdded
attendance.csv	SessionID, Date, RegNo, Name, Status (final FSM state), MarkedAt
audit_log.csv	Timestamp, SessionID, RegNo, From, Event, To — every FSM transition
reject_log.csv	Timestamp, Source, Input, Reason — every rejected input or blocked action
active_session.json	Snapshot of the running session (crash recovery)
settings.json	ID patterns, session length, on-time window

- **Validation before storage:** an ID reaches students.csv only if the DFA accepts it. Duplicates and invalid names (digits or symbols) are rejected.
- **Robust loading:** missing files are recreated. Rows with the wrong number of fields, bad values or duplicate IDs are moved to `students.quarantine-<time>.csv`, and the good rows load. A file with a wrong header is backed up and recreated.
- **Atomic writes:** whole-file saves go to a temporary file followed by `os.replace`, so a crash can never leave a half-written CSV.

9.3 Module 3 — Attendance Management (FSM)

Every student in a session owns one deterministic FSM. S = {NOT_MARKED, PRESENT, LATE, ABSENT, EXCUSED, ON_DUTY}, start state NOT_MARKED, E = {MARK_ON_TIME, MARK_LATE, TIMEOUT, EXCUSE, GRANT_OD, RESET}. The transition function $\delta : S \times E \rightarrow S$ is partial. An undefined pair raises InvalidTransition and is logged; it is never silently ignored.

State \ Event	MARK_ON_TIME	MARK_LATE	TIMEOUT	EXCUSE	GRANT_OD	RESET
NOT_MARKED	PRESENT	LATE	ABSENT	EXCUSED	ON_DUTY	—

State \ Event	MARK_ON_TIME	MARK_LATE	TIMEOUT	EXCUSE	GRANT_OD	RESET
PRESENT	—	—	—	—	—	NOT_MARKED
LATE	—	—	—	EXCUSED	ON_DUTY	NOT_MARKED
ABSENT	—	—	—	EXCUSED	ON_DUTY	—
EXCUSED	—	—	—	—	—	NOT_MARKED
ON_DUTY	—	—	—	—	—	NOT_MARKED

- **Separate start state.** NOT_MARKED is separate from ABSENT, so “not marked yet” and “absent” are different facts and TIMEOUT has a real effect.
- **Double marking is impossible.** δ(PRESENT, MARK_ON_TIME) is undefined, so a second mark (or a proxy) is rejected and recorded.
- **ON_DUTY** (VIT OD) and **EXCUSED** are admin overrides. They are allowed even after the session closes, so ABSENT can become EXCUSED.
- **RESET** (undo a wrong mark) is allowed only while the session is running.

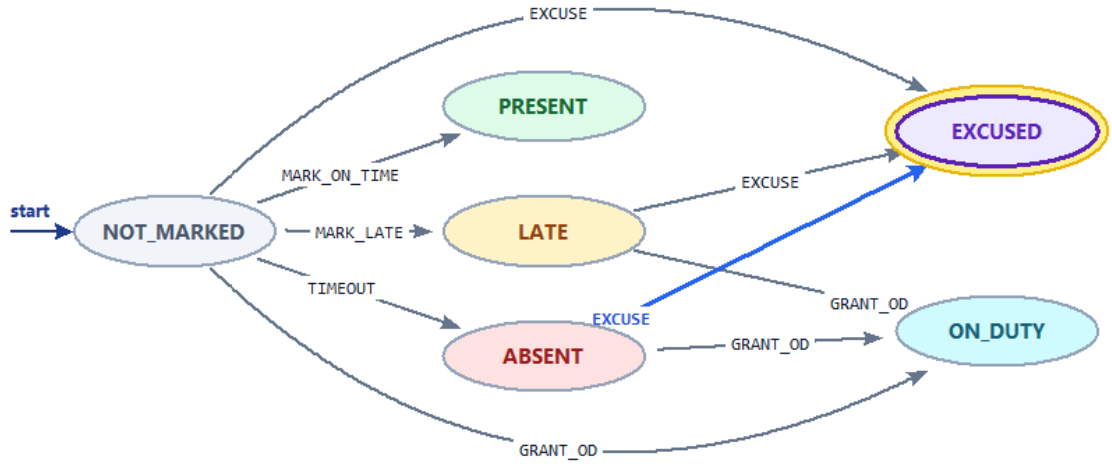


Figure 4: Attendance FSM drawn by the FSM visualizer. Highlighted: current state EXCUSED; blue arrow: the last transition ABSENT --EXCUSE--> EXCUSED.

9.4 Module 4 — Graphical User Interface

Tab	What it does
Register Students	Registration number checked live by the DFA (colour, path, mini diagram); add / search / delete; error and recovery banners
Mark Attendance	Start / end a session, countdown and phase badge, mark by reg. no., Excuse / Grant OD / Undo, colour-coded FSM states and counts
DFA Visualizer	Edit patterns and rebuild M; animated run on an input tape (Run / Step); formal definition, run trace, δ-table
FSM Visualizer	State diagram with the selected student's state and last move; δ table; transition history
Reports & Logs	75 % summary, Excel export, open data folder, list of rejected inputs and blocked actions

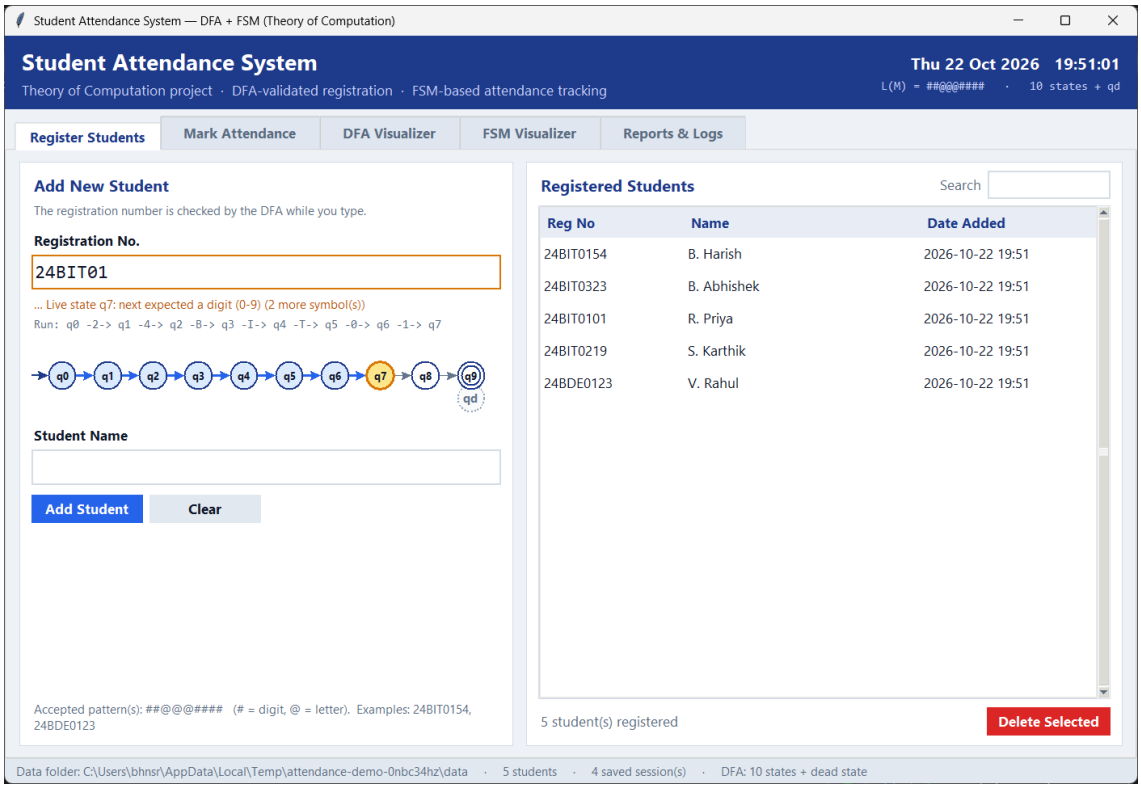


Figure 5: *Register tab: after typing 24BIT01 the DFA is in live state q7, which needs 2 more digits. The mini diagram highlights the path.*

9.5 Module 5 — Timer & Control

```
0 ............ on_time ................ duration
|  ON_TIME phase   |     LATE phase       |  CLOSED
mark → MARK_ON_TIME   mark → MARK_LATE      TIMEOUT for every NOT_MARKED student
```

- The **timer decides the event** and the **FSM decides the state**. The same action (marking) therefore gives PRESENT or LATE depending on the phase.
- When the window ends, TIMEOUT is fired automatically for every unmarked student (NOT_MARKED → ABSENT) and the final states are saved.
- Every transition updates `active_session.json`. If the app closes or crashes, the next start offers to restore the session with its timer.
- The clock is injectable, so tests can move time forward without waiting.

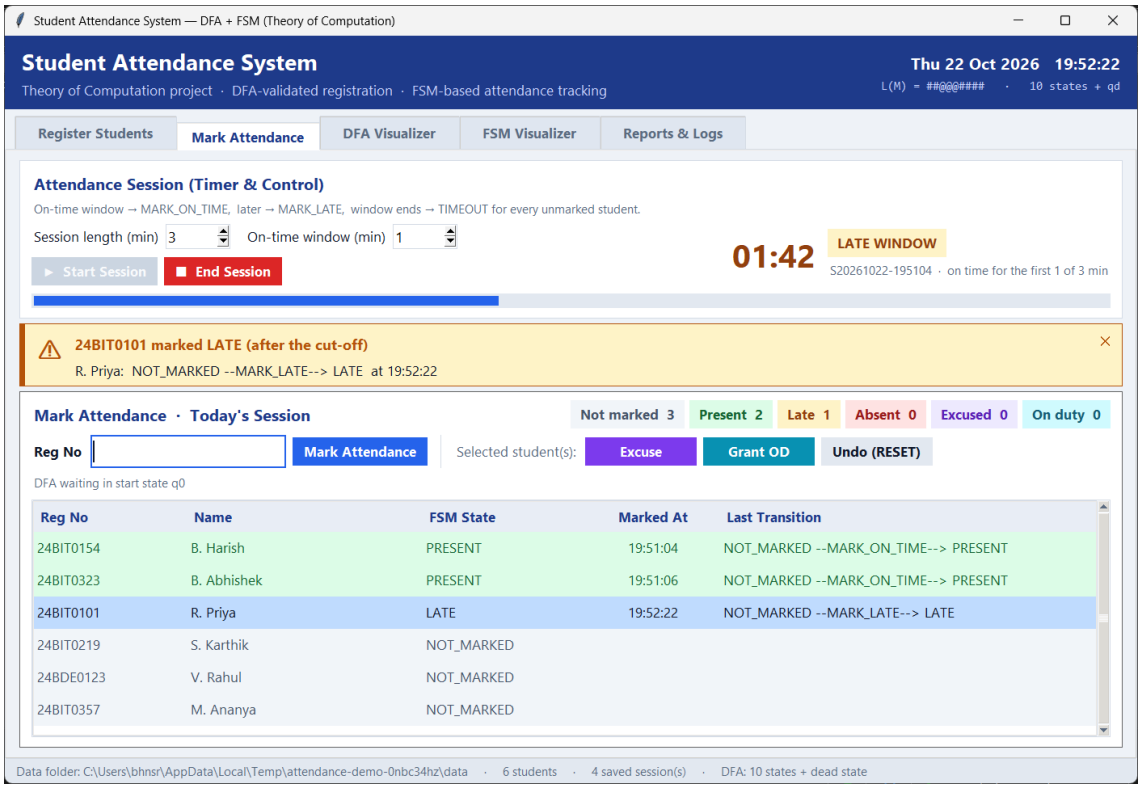


Figure 6: *After the on-time window the same action produces MARK_LATE, so R. Priya becomes LATE.*

9.6 Module 6 — Attendance Analytics (75 % rule)

```
attended = PRESENT + LATE + ON_DUTY
counted  = sessions held − EXCUSED
percent  = 100 × attended / counted
classes needed in a row to reach 75 %:  smallest x with (a + x)/(t + x) ≥ 0.75  ⇒  x = 3t − 4a
```

Example from the demo data: V. Rahul attended 2 of 5 classes (40 %), so he needs 3·5 − 4·2 = **7** classes in a row to get back to 75 %.

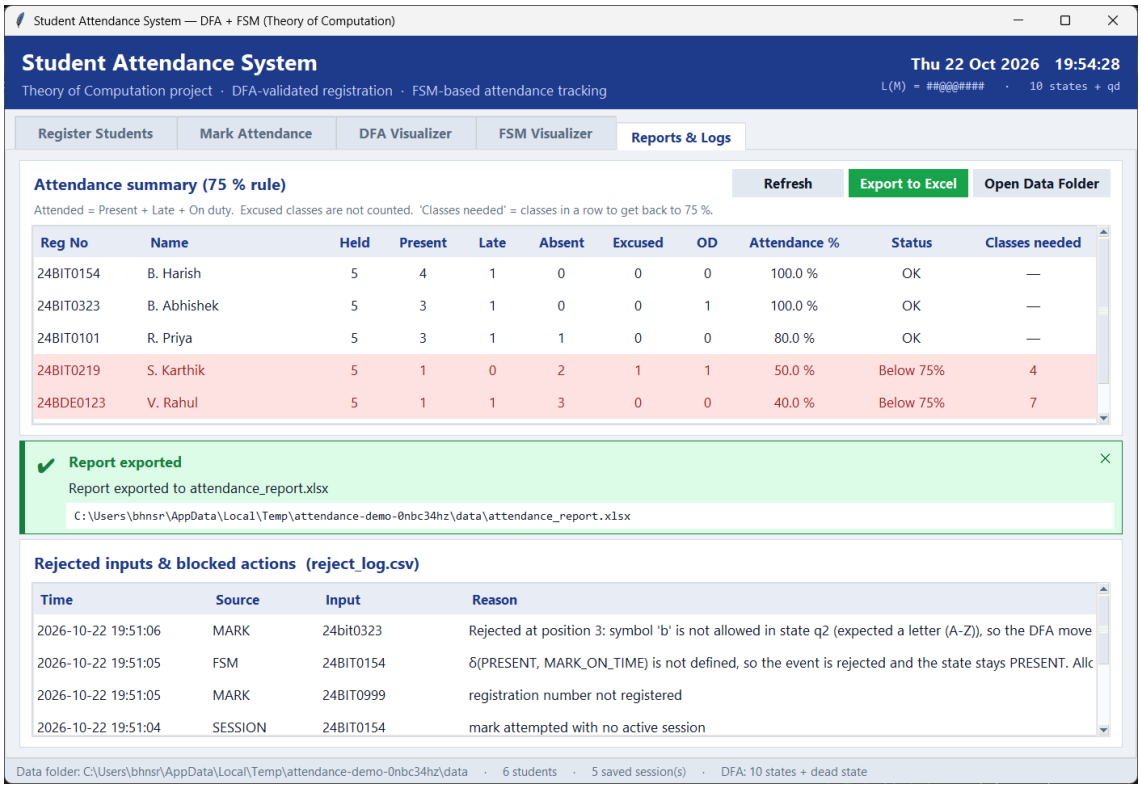


Figure 7: *Reports tab: the 75 % summary (rows below 75 % in red), Excel export, and the log of rejected inputs and blocked actions.*

9.7 Module 7 — Report Export

Exports an Excel workbook with three sheets: Summary (with red rows below 75 %), Attendance (colour-coded statuses) and Students. If openpyxl is not installed, the export does not fail. It writes three CSV files instead and says so (graceful degradation).

10. Role of Team Members

Member	Responsibilities
B. Harish (24BIT0154)	DFA design and implementation (Module 1): pattern compiler, subset construction, Moore minimisation, trace and explained rejections, DFA-guided correction. FSM state and transition design (Module 3). DFA Visualizer. Report compilation and GitHub repository.
B. Abhishek (24BIT0323)	GUI (Module 4): Tkinter screens and FSM Visualizer. Student database and storage (Module 2): CSV persistence, duplicate handling, quarantine and crash snapshot. Timer & Control (Module 5). Analytics and Excel export (Modules 6–7). Testing and screenshots.

Both members prepared the demo and the viva together.

11. Software Description

11.1 Overview

The system is a desktop application written in Python. The theory (DFA, FSM, timer) lives in small modules that have **no GUI code**, so they can be tested on their own. The Tkinter GUI only calls these modules and displays their results. Data is kept in plain CSV files that open in Excel.

11.2 Requirements

Item	Requirement
Operating system	Windows 10/11 (also runs on Linux / macOS with Tkinter)
Python	3.10 or newer (tested on 3.13)
Libraries	Standard library only for the core; openpyxl for Excel export; pytest for tests
Hardware	Any PC with 4 GB RAM; screen 1280×720 or larger

11.3 Installation and running

```
git clone https://github.com/HARISH0789/TOC-Student-Attendance-System.git
cd TOC-Student-Attendance-System
pip install -r requirements.txt
python main.py                    # data kept in ./data
python main.py --data-dir demo    # separate demo data
python -m pytest -q               # run the 67 tests
```

11.4 Project structure

```
TOC-Student-Attendance-System/
├── main.py                 start the application
├── attendance/
│   ├── dfa.py              Module 1  DFA compiler, trace, suggestions
│   ├── storage.py          Module 2  CSV storage and recovery
│   ├── fsm.py              Module 3  attendance FSM
│   ├── session.py          Module 5  timer & control
│   ├── analytics.py        Module 6  75 % rule
│   ├── exporter.py         Module 7  Excel / CSV export
│   └── gui/                Module 4  app.py, diagrams.py, widgets.py, theme.py
├── tests/                  test_dfa.py, test_fsm_session.py, test_storage_reports.py
├── tools/                  capture_screenshots.py, shot.py
└── docs/                   report and screenshots
```

11.5 Workflow

1. Faculty registers students. Each ID is validated by the DFA; corrections are suggested.
2. Faculty starts a session (for example 3 minutes, of which the first 1 minute is on time). Every student starts in NOT_MARKED.
3. Students are marked by registration number. The timer produces MARK_ON_TIME or MARK_LATE, and the FSM moves to PRESENT or LATE. Invalid or duplicate attempts are rejected and logged.
4. Faculty can Excuse, Grant OD or Undo (RESET) for selected students.
5. At the end of the window, TIMEOUT moves the remaining students to ABSENT. Results are saved to attendance.csv.
6. The Reports tab shows attendance % and the 75 % rule, and the report can be exported to Excel.

11.6 Testing

Test file	Tests	What is checked
test_dfa.py	34	Accept / reject cases, single-batch pattern, trace messages, suggestions, union + minimisation preserve the language, bad patterns
test_fsm_session.py	18	Valid event sequences, double marking rejected, phases of the timer, TIMEOUT, RESET rules, snapshot round trip
test_storage_reports.py	15	Files and headers, duplicates, names, quarantine of corrupted rows, bad header, corrupt snapshot, 75 % rule, Excel and CSV export
Total	67	All passing (python -m pytest -q)

12. Coding

The complete source code is in the GitHub repository: https://github.com/HARISH0789/TOC-Student-Attendance-System

The core modules are listed in full below, followed by the key GUI event handlers.

12.1 Module 1 — dfa.py

```
"""
Module 1 - Registration Number Validation using a DFA
=====================================================

A registration number is accepted only if it belongs to the regular
language L of valid IDs. L is recognised by a deterministic finite
automaton

        M = (Q, Sigma, delta, q0, F)

Pattern compiler
----------------
The DFA is not hard-coded. It is *compiled* from one or more patterns,
so a new ID format needs no code change:

    '#'  any digit  0-9
    '@'  any letter A-Z
    anything else is a literal symbol, e.g. 2 4 B I T

    "##@@@####"  ->  any VIT number such as 24BIT0154 or 24BDE0123
    "24BIT####"  ->  only the 2024 B.Tech IT batch

Several patterns describe the union L1 U L2 U ... . Each pattern is a
chain of NFA states; the union NFA is turned into a DFA with the subset
construction (Rabin & Scott, 1959) and then minimised with Moore's
partition-refinement algorithm.

Error recovery
--------------
When the DFA rejects an input we report *where* and *why* (state, symbol,
expected symbol class) and try a phrase-level correction guided by the
DFA: remove separators, fix lowercase, and swap look-alike symbols such as
'O' -> '0' or 'S' -> '5' only when the DFA says the swap is legal.
"""

from __future__ import annotations

import string
from collections import deque
from dataclasses import dataclass, field

DIGITS = frozenset(string.digits)
LETTERS = frozenset(string.ascii_uppercase)
ALPHABET = tuple(sorted(DIGITS) + sorted(LETTERS))  # Sigma (36 symbols)
DEAD = "qd"  # dead / trap state: every missing transition goes here
DEFAULT_PATTERNS = ("##@@@####",)

SEPARATORS = set(" \t-_/.")
LOOKS_LIKE_DIGIT = {"O": "0", "Q": "0", "D": "0", "I": "1", "L": "1",
                    "Z": "2", "S": "5", "G": "6", "T": "7", "B": "8"}
LOOKS_LIKE_LETTER = {"0": "O", "1": "I", "2": "Z", "4": "A", "5": "S",
                     "6": "G", "7": "T", "8": "B"}


class PatternError(ValueError):
    """Raised when an ID pattern cannot be compiled into a DFA."""


# ---------------------------------------------------------------------------
# Helpers for readable symbol sets
# ---------------------------------------------------------------------------
def compress(symbols) -> str:
    """'0123456789' -> '0-9'  and  'ABCDEFGHJ' -> 'A-H,J'."""
    order = [c for c in ALPHABET if c in symbols]
    parts, i = [], 0
    while i < len(order):
        j = i
        while j + 1 < len(order) and ord(order[j + 1]) == ord(order[j]) + 1:
```

```
            j += 1
        if j - i >= 2:
            parts.append(f"{order[i]}-{order[j]}")
        else:
            parts.extend(order[i:j + 1])
        i = j + 1
    return ",".join(parts)


def edge_label(symbols) -> str:
    s = frozenset(symbols)
    if s == DIGITS:
        return "0-9"
    if s == LETTERS:
        return "A-Z"
    return compress(s)


def describe(symbols) -> str:
    """Human description of what a state expects next."""
    s = frozenset(symbols)
    if not s:
        return "nothing more (the number is complete)"
    if s == DIGITS:
        return "a digit (0-9)"
    if s == LETTERS:
        return "a letter (A-Z)"
    if s == DIGITS | LETTERS:
        return "a digit or a letter"
    if len(s) == 1:
        return f"'{next(iter(s))}'"
    return "one of " + compress(s)


def parse_pattern(pattern: str) -> list[frozenset]:
    """Turn '24BIT####' into a list with one allowed-symbol set per position."""
    pattern = pattern.strip().upper()
    if not pattern:
        raise PatternError("A pattern cannot be empty.")
    chain = []
    for pos, ch in enumerate(pattern, start=1):
        if ch == "#":
            chain.append(DIGITS)
        elif ch == "@":
            chain.append(LETTERS)
        elif ch in DIGITS or ch in LETTERS:
            chain.append(frozenset(ch))
        else:
            raise PatternError(
                f"Unsupported character '{ch}' at position {pos} of pattern "
                f"'{pattern}'. Use # for a digit, @ for a letter, or A-Z / 0-9 literals.")
    return chain


# ---------------------------------------------------------------------------
# Results returned to the GUI
# ---------------------------------------------------------------------------
@dataclass
class Step:
    position: int  # 0-based index of the symbol in the input
    symbol: str
    source: str
    target: str

    def __str__(self) -> str:
        return f"δ({self.source}, '{self.symbol}') = {self.target}"


@dataclass
class Trace:
    text: str
    steps: list = field(default_factory=list)
    accepted: bool = False
    final_state: str = "q0"
    error_position: int | None = None  # 0-based index of the offending symbol
    reason: str = ""

    @property
    def path(self) -> list[str]:
        if not self.steps:
            return ["q0"]
```

```
        return [self.steps[0].source] + [s.target for s in self.steps]

    def path_text(self) -> str:
        if not self.steps:
            return "q0"
        out = self.steps[0].source
        for s in self.steps:
            out += f" -{s.symbol}-> {s.target}"
        return out


@dataclass
class Suggestion:
    original: str
    corrected: str
    fixes: list = field(default_factory=list)


# ---------------------------------------------------------------------
# The DFA
# ---------------------------------------------------------------------
class RegNoDFA:
    """Table-driven DFA compiled from one or more ID patterns."""

    def __init__(self, patterns=DEFAULT_PATTERNS, minimize: bool = True):
        if isinstance(patterns, str):
            patterns = [p for p in patterns.split(",")]
        cleaned = []
        for p in patterns:
            p = p.strip().upper()
            if p and p not in cleaned:
                cleaned.append(p)
        if not cleaned:
            raise PatternError("Enter at least one pattern, for example ##@@#####.")
        chains = [parse_pattern(p) for p in cleaned]
        self.patterns = tuple(cleaned)

        states, delta, accept = _subset_construction(chains)
        self.states_before_minimization = len(states)
        if minimize:
            states, delta, accept = _minimize(states, delta, accept)
        self.states: list[str] = states  # live states, q0 first (dead state not listed)
        self.delta: dict[str, dict[str, str]] = delta
        self.accept: frozenset = accept
        self.start = "q0"

    # --- basic DFA operations ----------------------------------------
    def next_state(self, state: str, symbol: str) -> str:
        if state == DEAD:
            return DEAD
        return self.delta[state].get(symbol, DEAD)

    def expected(self, state: str) -> frozenset:
        if state == DEAD:
            return frozenset()
        return frozenset(self.delta[state])

    def distance_to_accept(self, state: str) -> int | None:
        """Fewest symbols still needed to reach a final state (BFS)."""
        if state == DEAD:
            return None
        seen, queue = {state}, deque([(state, 0)])
        while queue:
            q, d = queue.popleft()
            if q in self.accept:
                return d
            for t in set(self.delta[q].values()):
                if t not in seen:
                    seen.add(t)
                    queue.append((t, d + 1))
        return None

    def trace(self, text: str) -> Trace:
        """Run the DFA and record every transition (used by GUI and errors)."""
        tr = Trace(text=text)
        state = self.start
        if text == "":
            tr.final_state = state
            tr.reason = ("Input is empty. The DFA is still in the start state q0, "
                         "which is not a final state.")
            return tr
```

```
        for i, ch in enumerate(text):
            nxt = self.next_state(state, ch)
            tr.steps.append(Step(i, ch, state, nxt))
            if nxt == DEAD:
                tr.final_state = DEAD
                tr.error_position = i
                exp = self.expected(state)
                if not exp:
                    tr.reason = (f"Rejected at position {i + 1}: the number is already complete "
                                 f"in final state {state}, so the extra symbol '{ch}' leads to "
                                 f"the dead state {DEAD}.")
                else:
                    hint = ""
                    if ch.islower():
                        hint = " Lowercase letters are not in Σ."
                    elif ch in SEPARATORS:
                        hint = " Spaces and separators are not in Σ."
                    tr.reason = (f"Rejected at position {i + 1}: symbol '{ch}' is not allowed in "
                                 f"state {state} (expected {describe(exp)}), so the DFA moves "
                                 f"to the dead state {DEAD}.{hint}")
                return tr
            state = nxt
        tr.final_state = state
        if state in self.accept:
            tr.accepted = True
            tr.reason = f"Accepted: the run ends in final state {state} ∈ F."
        else:
            need = self.distance_to_accept(state)
            tr.reason = (f"Rejected: the input ended in state {state}, which is not a final "
                         f"state. {need} more symbol(s) needed; next expected "
                         f"{describe(self.expected(state))}.")
        return tr

    def accepts(self, text: str) -> bool:
        return self.trace(text).accepted

    validate = accepts  # alias, e.g. validate("24BIT0154") -> True

    # --- error recovery ------------------------------------------------
    def suggest(self, text: str) -> Suggestion | None:
        """Phrase-level recovery: return a corrected ID if a safe fix exists."""
        fixes = []
        cleaned = "".join(ch for ch in text if ch not in SEPARATORS)
        if cleaned != text:
            fixes.append("removed spaces / separators")
        upper = cleaned.upper()
        if upper != cleaned:
            fixes.append("converted lowercase letters to uppercase")
        state, out = self.start, []
        for i, ch in enumerate(upper):
            nxt = self.next_state(state, ch)
            if nxt == DEAD:
                for cand in (LOOKS_LIKE_DIGIT.get(ch), LOOKS_LIKE_LETTER.get(ch)):
                    if cand and self.next_state(state, cand) != DEAD:
                        fixes.append(f"position {i + 1}: '{ch}' → '{cand}' "
                                     f"(state {state} expects {describe(self.expected(state))})")
                        ch, nxt = cand, self.next_state(state, cand)
                        break
                else:
                    return None  # no safe correction exists
            out.append(ch)
            state = nxt
        corrected = "".join(out)
        if state in self.accept and corrected != text:
            return Suggestion(text, corrected, fixes)
        return None

    # --- views used by the visualizer and the report -------------------
    def edges(self) -> list[tuple[str, str, str]]:
        """(source, label, target) with symbols grouped per target state."""
        result = []
        for q in self.states:
            groups: dict[str, set] = {}
            for sym, t in self.delta[q].items():
                groups.setdefault(t, set()).add(sym)
            for t, syms in groups.items():
                result.append((q, edge_label(syms), t))
        return result

    def levels(self) -> dict[str, int]:
        """BFS depth of each state (patterns are fixed length, so M is acyclic)."""
```

```
        depth, queue = {self.start: 0}, deque([self.start])
        while queue:
            q = queue.popleft()
            for t in self.delta[q].values():
                if t not in depth:
                    depth[t] = depth[q] + 1
                    queue.append(t)
        return depth

    def transition_rows(self) -> list[tuple[str, str, str]]:
        rows = []
        for q in self.states:
            kind = []
            if q == self.start:
                kind.append("start")
            if q in self.accept:
                kind.append("final")
            moves = [f"{lbl} → {t}" for s, lbl, t in self.edges() if s == q]
            rows.append((q, ", ".join(kind) or "-", "; ".join(moves) or "(none: all → qd)"))
        rows.append((DEAD, "dead", "every symbol → qd"))
        return rows

    def formal_definition(self) -> str:
        if len(self.states) > 5:
            names = f"{self.states[0]}, {self.states[1]}, …, {self.states[-1]}"
        else:
            names = ", ".join(self.states)
        return (f"Q  = {{{names}, {DEAD}}}  ({len(self.states) + 1} states)\n"
                f"Σ  = {{0-9, A-Z}}  (36 symbols)\n"
                f"q0 = {self.start}\n"
                f"F  = {{{', '.join(sorted(self.accept, key=_state_key))}}}\n"
                f"δ  = transition table (missing entries → {DEAD})\n"
                f"L(M) = {' ∪ '.join(self.patterns)}")


def _state_key(name: str) -> int:
    return int(name[1:]) if name[1:].isdigit() else 10 ** 6


def _subset_construction(chains):
    """NFA state = (pattern index, position). DFA state = set of NFA states."""
    start = frozenset((i, 0) for i in range(len(chains)))
    names = {start: "q0"}
    order, queue = [start], deque([start])
    delta, accept = {}, set()
    while queue:
        subset = queue.popleft()
        name = names[subset]
        if any(pos == len(chains[i]) for i, pos in subset):
            accept.add(name)
        row = {}
        for sym in ALPHABET:
            target = frozenset((i, pos + 1) for i, pos in subset
                               if pos < len(chains[i]) and sym in chains[i][pos])
            if not target:
                continue  # empty set = dead state, left implicit
            if target not in names:
                names[target] = f"q{len(names)}"
                order.append(target)
                queue.append(target)
            row[sym] = names[target]
        delta[name] = row
    return [names[s] for s in order], delta, frozenset(accept)


def _minimize(states, delta, accept):
    """Moore's algorithm: split blocks until no symbol can tell states apart."""
    everything = list(states) + [DEAD]

    def step(q, a):
        return DEAD if q == DEAD else delta[q].get(a, DEAD)

    block = {q: (1 if q in accept else 0) for q in everything}
    while True:
        signatures: dict[tuple, int] = {}
        new_block = {}
        for q in everything:
            sig = (block[q],) + tuple(block[step(q, a)] for a in ALPHABET)
            new_block[q] = signatures.setdefault(sig, len(signatures))
        stable = len(signatures) == len(set(block.values()))
        block = new_block
```

```
        if stable:
            break

    dead_block = block[DEAD]
    representative = {}
    for q in everything:
        representative.setdefault(block[q], q)
    start_block = block["q0"]
    names = {start_block: "q0"}
    order, queue = [start_block], deque([start_block])
    new_delta, new_accept = {}, set()
    while queue:
        b = queue.popleft()
        q, name = representative[b], names[b]
        if q in accept:
            new_accept.add(name)
        row = {}
        for a in ALPHABET:
            t = block[step(q, a)]
            if t == dead_block:
                continue
            if t not in names:
                names[t] = f"q{len(names)}"
                order.append(t)
                queue.append(t)
            row[a] = names[t]
        new_delta[name] = row
    return [names[b] for b in order], new_delta, frozenset(new_accept)
```

12.2 Module 3 — fsm.py

```
"""
Module 3 - Attendance Management using a Finite State Machine
============================================================

Every student in a session owns one FSM. The machine is always in exactly
one state and only moves when an event arrives:

    States  S = {NOT_MARKED, PRESENT, LATE, ABSENT, EXCUSED, ON_DUTY}
    Start     = NOT_MARKED
    Events  E = {MARK_ON_TIME, MARK_LATE, TIMEOUT, EXCUSE, GRANT_OD, RESET}
    delta : S x E -> S   (partial function, see TRANSITIONS)

Design notes
------------
* NOT_MARKED is a separate start state, so "not marked yet" and "absent"
  are different states and the time-out has a real effect.
* An ON_DUTY state (VIT "OD") reached through GRANT_OD.
* RESET lets the faculty undo a wrong mark while the session is running.
* Strict machine: an undefined (state, event) pair raises InvalidTransition
  instead of being silently ignored. This is what blocks double marking.
* Every transition is recorded (history / audit trail).
"""

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime

NOT_MARKED = "NOT_MARKED"
PRESENT = "PRESENT"
LATE = "LATE"
ABSENT = "ABSENT"
EXCUSED = "EXCUSED"
ON_DUTY = "ON_DUTY"
STATES = (NOT_MARKED, PRESENT, LATE, ABSENT, EXCUSED, ON_DUTY)
START_STATE = NOT_MARKED

MARK_ON_TIME = "MARK_ON_TIME"
MARK_LATE = "MARK_LATE"
TIMEOUT = "TIMEOUT"
EXCUSE = "EXCUSE"
GRANT_OD = "GRANT_OD"
RESET = "RESET"
EVENTS = (MARK_ON_TIME, MARK_LATE, TIMEOUT, EXCUSE, GRANT_OD, RESET)

# States that count as "attended" for the 75 % rule.
ATTENDED = frozenset({PRESENT, LATE, ON_DUTY})

TRANSITIONS = {
```

```
    (NOT_MARKED, MARK_ON_TIME): PRESENT,
    (NOT_MARKED, MARK_LATE): LATE,
    (NOT_MARKED, TIMEOUT): ABSENT,
    (NOT_MARKED, EXCUSE): EXCUSED,
    (NOT_MARKED, GRANT_OD): ON_DUTY,
    (LATE, EXCUSE): EXCUSED,
    (LATE, GRANT_OD): ON_DUTY,
    (ABSENT, EXCUSE): EXCUSED,
    (ABSENT, GRANT_OD): ON_DUTY,
    # admin undo (the session layer only allows it while the session runs)
    (PRESENT, RESET): NOT_MARKED,
    (LATE, RESET): NOT_MARKED,
    (EXCUSED, RESET): NOT_MARKED,
    (ON_DUTY, RESET): NOT_MARKED,
}


def allowed_events(state: str) -> list[str]:
    return [e for e in EVENTS if (state, e) in TRANSITIONS]


class InvalidTransition(Exception):
    """delta(state, event) is undefined, so the event is rejected."""

    def __init__(self, state: str, event: str):
        self.state = state
        self.event = event
        self.allowed = allowed_events(state)
        allowed = ", ".join(self.allowed) if self.allowed else "none"
        super().__init__(f"δ({state}, {event}) is not defined, so the event is rejected "
                         f"and the state stays {state}. Allowed events from {state}: {allowed}.")


@dataclass(frozen=True)
class TransitionRecord:
    time: str
    source: str
    event: str
    target: str

    def __str__(self) -> str:
        return f"{self.time}  {self.source} --{self.event}--> {self.target}"


class AttendanceFSM:
    def __init__(self, state: str = START_STATE):
        if state not in STATES:
            raise ValueError(f"Unknown state {state!r}")
        self.state = state
        self.history: list[TransitionRecord] = []

    def can_fire(self, event: str) -> bool:
        return (self.state, event) in TRANSITIONS

    def allowed_events(self) -> list[str]:
        return allowed_events(self.state)

    def fire(self, event: str, when: datetime | None = None) -> TransitionRecord:
        if event not in EVENTS:
            raise ValueError(f"Unknown event {event!r}")
        key = (self.state, event)
        if key not in TRANSITIONS:
            raise InvalidTransition(self.state, event)
        stamp = (when or datetime.now()).strftime("%H:%M:%S")
        record = TransitionRecord(stamp, self.state, event, TRANSITIONS[key])
        self.state = record.target
        self.history.append(record)
        return record

    @property
    def is_marked(self) -> bool:
        return self.state != NOT_MARKED
```

12.3 Module 5 — session.py

```
"""
Module 5 - Timer & Control
==========================

An attendance session is a fixed time window, split into two phases:
```

```
    0 ............ on_time ................ duration
    |  ON_TIME phase   |    LATE phase        |  CLOSED
    MARK -> MARK_ON_TIME  MARK -> MARK_LATE     TIMEOUT for everyone unmarked

The timer decides *which event* a "mark" produces; the FSM decides the
resulting state. When the window ends, every student still in NOT_MARKED
receives the TIMEOUT event and becomes ABSENT.

The clock is injectable so the timer can be tested without waiting, and the
whole session can be saved to / restored from a dict (crash recovery).
"""

from __future__ import annotations

import time
from datetime import datetime

from .fsm import (EXCUSE, GRANT_OD, MARK_LATE, MARK_ON_TIME, NOT_MARKED, RESET, STATES,
                  TIMEOUT, AttendanceFSM, TransitionRecord)

ON_TIME = "ON_TIME"
LATE_PHASE = "LATE"
CLOSED = "CLOSED"


class SessionError(Exception):
    pass


class SessionClosed(SessionError):
    pass


class UnknownStudent(SessionError):
    pass


class AttendanceSession:
    def __init__(self, roster: dict[str, str], duration_sec: float = 180,
                 on_time_sec: float = 60, clock=time.time,
                 session_id: str | None = None, started_at: float | None = None):
        if duration_sec <= 0:
            raise ValueError("Session length must be greater than zero.")
        if not 0 < on_time_sec <= duration_sec:
            raise ValueError("The on-time window must be greater than zero and not "
                             "longer than the session.")
        self.clock = clock
        self.started_at = clock() if started_at is None else started_at
        self.session_id = session_id or datetime.fromtimestamp(self.started_at).strftime(
            "S%Y%m%d-%H%M%S")
        self.duration = float(duration_sec)
        self.on_time = float(on_time_sec)
        self.roster = dict(roster)
        self.machines = {reg: AttendanceFSM() for reg in self.roster}
        self.marked_at: dict[str, str] = {}
        self.closed = False
        self.ended_at: float | None = None

    # --- timer ------------------------------------------------------
    def _now(self) -> datetime:
        return datetime.fromtimestamp(self.clock())

    def elapsed(self) -> float:
        end = self.ended_at if self.closed and self.ended_at else self.clock()
        return max(0.0, end - self.started_at)

    def remaining(self) -> float:
        return 0.0 if self.closed else max(0.0, self.duration - self.elapsed())

    def phase(self) -> str:
        if self.closed or self.elapsed() >= self.duration:
            return CLOSED
        return ON_TIME if self.elapsed() < self.on_time else LATE_PHASE

    @property
    def expired(self) -> bool:
        return not self.closed and self.elapsed() >= self.duration

    # --- events -----------------------------------------------------
    def machine(self, regno: str) -> AttendanceFSM:
```

```
        if regno not in self.machines:
            raise UnknownStudent(f"{regno} is not part of session {self.session_id}.")
        return self.machines[regno]

    def add_student(self, regno: str, name: str) -> None:
        if regno not in self.machines:
            self.roster[regno] = name
            self.machines[regno] = AttendanceFSM()

    def mark(self, regno: str) -> TransitionRecord:
        """Student marks attendance; the current phase picks the event."""
        if self.phase() == CLOSED:
            raise SessionClosed("The attendance window has closed. Start a new session to mark again.")
        fsm = self.machine(regno)
        event = MARK_ON_TIME if self.phase() == ON_TIME else MARK_LATE
        record = fsm.fire(event, self._now())
        self.marked_at[regno] = record.time
        return record

    def excuse(self, regno: str) -> TransitionRecord:
        return self.machine(regno).fire(EXCUSE, self._now())

    def grant_od(self, regno: str) -> TransitionRecord:
        return self.machine(regno).fire(GRANT_OD, self._now())

    def reset(self, regno: str) -> TransitionRecord:
        if self.phase() == CLOSED:
            raise SessionClosed("RESET (undo) is only allowed while the session is running.")
        record = self.machine(regno).fire(RESET, self._now())
        self.marked_at.pop(regno, None)
        return record

    def close(self) -> list[tuple[str, TransitionRecord]]:
        """End the window: TIMEOUT moves every NOT_MARKED student to ABSENT."""
        if self.closed:
            return []
        now = self._now()
        if self.expired:  # use the exact end of the window, not the moment we noticed
            now = datetime.fromtimestamp(self.started_at + self.duration)
        records = []
        for regno, fsm in self.machines.items():
            if fsm.state == NOT_MARKED:
                records.append((regno, fsm.fire(TIMEOUT, now)))
        self.ended_at = min(self.clock(), self.started_at + self.duration)
        self.closed = True
        return records

    # --- views ------------------------------------------------------------
    def counts(self) -> dict[str, int]:
        result = {s: 0 for s in STATES}
        for fsm in self.machines.values():
            result[fsm.state] += 1
        return result

    def rows(self) -> list[dict]:
        out = []
        for regno, fsm in self.machines.items():
            last = fsm.history[-1] if fsm.history else None
            out.append({
                "regno": regno,
                "name": self.roster.get(regno, ""),
                "state": fsm.state,
                "marked_at": self.marked_at.get(regno, ""),
                "last": f"{last.source} --{last.event}--> {last.target}" if last else "",
            })
        return out

    # --- crash recovery ---------------------------------------------------
    def to_dict(self) -> dict:
        return {
            "version": 1,
            "session_id": self.session_id,
            "started_at": self.started_at,
            "duration": self.duration,
            "on_time": self.on_time,
            "roster": self.roster,
            "states": {r: m.state for r, m in self.machines.items()},
            "marked_at": self.marked_at,
            "history": {r: [[h.time, h.source, h.event, h.target] for h in m.history]
                        for r, m in self.machines.items()},
        }
```

```
    @classmethod
    def from_dict(cls, data: dict, clock=time.time) -> "AttendanceSession":
        session = cls(data["roster"], data["duration"], data["on_time"], clock,
                      data["session_id"], data["started_at"])
        for regno, state in data["states"].items():
            if regno in session.machines:
                if state not in STATES:
                    raise ValueError(f"Unknown state {state!r} in saved session")
                session.machines[regno].state = state
                session.machines[regno].history = [
                    TransitionRecord(*h) for h in data.get("history", {}).get(regno, [])]
        session.marked_at = dict(data.get("marked_at", {}))
        return session
```

12.4 Module 2 — storage.py

```
"""
Module 2 - Student Database & Data Storage (CSV)
================================================

Files kept in the data folder:

    students.csv        RegNo, Name, DateAdded
    attendance.csv      SessionID, Date, RegNo, Name, Status, MarkedAt
    audit_log.csv       every FSM transition (who, from, event, to, when)
    reject_log.csv      every rejected input (DFA rejects, invalid transitions ...)
    active_session.json snapshot of a running session (crash recovery)
    settings.json       ID patterns and timer settings

Error recovery
--------------
* Missing files are recreated with the right header.
* Malformed rows are moved to a "quarantine" CSV and the good rows are kept,
  so one bad line never stops the app from starting.
* A file with a wrong header or unreadable bytes is backed up and recreated.
* Whole-file rewrites go through a temp file + os.replace (atomic), so a crash
  in the middle of a save cannot leave a half-written CSV behind.
"""

from __future__ import annotations

import csv
import json
import os
import re
import shutil
import tempfile
import time
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path

from .dfa import DEFAULT_PATTERNS, RegNoDFA, Suggestion, Trace
from .fsm import STATES

STUDENT_FIELDS = ["RegNo", "Name", "DateAdded"]
ATTENDANCE_FIELDS = ["SessionID", "Date", "RegNo", "Name", "Status", "MarkedAt"]
AUDIT_FIELDS = ["Timestamp", "SessionID", "RegNo", "From", "Event", "To"]
REJECT_FIELDS = ["Timestamp", "Source", "Input", "Reason"]

NAME_RE = re.compile(r"[A-Za-z][A-Za-z .']{1,49}")
DEFAULT_SETTINGS = {"patterns": list(DEFAULT_PATTERNS), "duration_min": 3.0, "on_time_min": 1.0}


class StoreError(Exception):
    pass


class InvalidRegNo(StoreError):
    def __init__(self, trace: Trace, suggestion: Suggestion | None):
        self.trace = trace
        self.suggestion = suggestion
        super().__init__(trace.reason)


class InvalidName(StoreError):
    pass
```

```
class DuplicateStudent(StoreError):
    def __init__(self, regno: str, name: str, added: str):
        self.regno, self.name, self.added = regno, name, added
        super().__init__(f"{regno} is already registered as {name} (added {added}).")


class StudentNotFound(StoreError):
    pass


@dataclass
class Student:
    regno: str
    name: str
    added: str


@dataclass
class LoadIssue:
    file: str
    good_rows: int
    bad_rows: int
    backup: str
    note: str


def now_stamp() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")


def _replace_with_retry(src: str, dst: Path) -> None:
    # OneDrive / antivirus can hold a file for a moment on Windows.
    for attempt in range(5):
        try:
            os.replace(src, dst)
            return
        except PermissionError:
            if attempt == 4:
                raise
            time.sleep(0.1)


def atomic_write_csv(path: Path, header: list[str], rows) -> None:
    path = Path(path)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=f".{path.stem}-", suffix=".tmp")
    with os.fdopen(fd, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)
    _replace_with_retry(tmp, path)


def atomic_write_json(path: Path, data) -> None:
    path = Path(path)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=f".{path.stem}-", suffix=".tmp")
    with os.fdopen(fd, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    _replace_with_retry(tmp, path)


class AttendanceStore:
    def __init__(self, data_dir, dfa: RegNoDFA | None = None):
        self.dir = Path(data_dir)
        self.dir.mkdir(parents=True, exist_ok=True)
        self.dfa = dfa or RegNoDFA()
        self.students_csv = self.dir / "students.csv"
        self.attendance_csv = self.dir / "attendance.csv"
        self.audit_csv = self.dir / "audit_log.csv"
        self.reject_csv = self.dir / "reject_log.csv"
        self.snapshot_json = self.dir / "active_session.json"
        self.settings_json = self.dir / "settings.json"
        self.issues: list[LoadIssue] = []
        self.students: dict[str, Student] = {}

        seen: set[str] = set()

        def student_ok(row):
            regno, name = row[0].strip(), row[1].strip()
            if not re.fullmatch(r"[A-Z0-9]{4,20}", regno) or not name or regno in seen:
                return False
            seen.add(regno)
```

```
            return True

        for row in self._load(self.students_csv, STUDENT_FIELDS, student_ok):
            self.students[row[0].strip()] = Student(row[0].strip(), row[1].strip(), row[2].strip())
        self._load(self.attendance_csv, ATTENDANCE_FIELDS,
                   lambda r: bool(r[0].strip()) and bool(r[2].strip()) and r[4] in STATES)
        self._load(self.audit_csv, AUDIT_FIELDS, lambda r: True)
        self._load(self.reject_csv, REJECT_FIELDS, lambda r: True)

    # --- robust loading ------------------------------------------------
    def _backup_name(self, path: Path, tag: str) -> Path:
        return self.dir / f"{path.stem}.{tag}-{datetime.now():%Y%m%d-%H%M%S}{path.suffix}"

    def _load(self, path: Path, fields: list[str], row_ok) -> list[list[str]]:
        if not path.exists():
            atomic_write_csv(path, fields, [])
            return []
        try:
            with open(path, newline="", encoding="utf-8") as f:
                rows = list(csv.reader(f))
        except (UnicodeDecodeError, csv.Error):
            rows = None
        if rows is not None and not rows:  # empty file: just add the header
            atomic_write_csv(path, fields, [])
            return []
        if rows is None or [c.strip() for c in rows[0]] != fields:
            backup = self._backup_name(path, "corrupt")
            shutil.copyfile(path, backup)
            atomic_write_csv(path, fields, [])
            self.issues.append(LoadIssue(path.name, 0, 0, backup.name,
                                         "header missing or file unreadable; the file was backed "
                                         "up and recreated"))
            return []
        good, bad = [], []
        for row in rows[1:]:
            if not row or all(not c.strip() for c in row):
                continue  # ignore blank lines
            if len(row) == len(fields) and row_ok(row):
                good.append(row)
            else:
                bad.append(row)
        if bad:
            backup = self._backup_name(path, "quarantine")
            with open(backup, "w", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                writer.writerow([f"# rows removed from {path.name} on {now_stamp()}"])
                writer.writerows(bad)
            atomic_write_csv(path, fields, good)
            self.issues.append(LoadIssue(path.name, len(good), len(bad), backup.name,
                                         "malformed rows were moved to quarantine"))
        return good

    @staticmethod
    def _append(path: Path, row: list) -> None:
        with open(path, "a", newline="", encoding="utf-8") as f:
            csv.writer(f).writerow(row)

    @staticmethod
    def _dict_rows(path: Path) -> list[dict]:
        try:
            with open(path, newline="", encoding="utf-8") as f:
                return list(csv.DictReader(f))
        except FileNotFoundError:
            return []

    # --- students ------------------------------------------------------
    def add_student(self, regno_raw: str, name_raw: str) -> Student:
        regno = regno_raw.strip()
        trace = self.dfa.trace(regno)
        if not trace.accepted:
            self.log_reject("REGISTER", regno, trace.reason)
            raise InvalidRegNo(trace, self.dfa.suggest(regno))
        if regno in self.students:
            s = self.students[regno]
            self.log_reject("REGISTER", regno, "duplicate registration blocked")
            raise DuplicateStudent(s.regno, s.name, s.added)
        name = " ".join(name_raw.split())
        if not name:
            raise InvalidName("Student name is required.")
        if not NAME_RE.fullmatch(name):
            bad = next((c for c in name if not (c.isalpha() or c in " .'")), None)
```

```
            detail = f" (found '{bad}')" if bad else ""
            raise InvalidName("Name must be 2-50 characters and may contain only letters, "
                              f"spaces, dots and apostrophes{detail}.")
        student = Student(regno, name, datetime.now().strftime("%Y-%m-%d %H:%M"))
        self.students[regno] = student
        self._append(self.students_csv, [student.regno, student.name, student.added])
        return student

    def delete_student(self, regno: str) -> Student:
        if regno not in self.students:
            raise StudentNotFound(f"{regno} is not registered.")
        student = self.students.pop(regno)
        atomic_write_csv(self.students_csv, STUDENT_FIELDS,
                         [[s.regno, s.name, s.added] for s in self.students.values()])
        return student

    # --- attendance ---------------------------------------------------------
    def save_session(self, session) -> None:
        """Write the final state of every student (replaces earlier rows of the session)."""
        rows = [[r[f] for f in ATTENDANCE_FIELDS] for r in self.attendance_rows()
                if r["SessionID"] != session.session_id]
        date = datetime.fromtimestamp(session.started_at).strftime("%Y-%m-%d")
        for regno, fsm in session.machines.items():
            rows.append([session.session_id, date, regno, session.roster.get(regno, ""),
                         fsm.state, session.marked_at.get(regno, "")])
        atomic_write_csv(self.attendance_csv, ATTENDANCE_FIELDS, rows)

    def attendance_rows(self) -> list[dict]:
        return self._dict_rows(self.attendance_csv)

    def session_ids(self) -> list[str]:
        return sorted({r["SessionID"] for r in self.attendance_rows()})

    # --- logs ---------------------------------------------------------------
    def log_transition(self, session_id: str, regno: str, record) -> None:
        self._append(self.audit_csv, [now_stamp(), session_id, regno,
                                      record.source, record.event, record.target])

    def log_reject(self, source: str, text: str, reason: str) -> None:
        self._append(self.reject_csv, [now_stamp(), source, text, reason])

    def reject_rows(self) -> list[dict]:
        return list(reversed(self._dict_rows(self.reject_csv)))

    def audit_rows(self) -> list[dict]:
        return list(reversed(self._dict_rows(self.audit_csv)))

    # --- crash-recovery snapshot --------------------------------------------
    def write_snapshot(self, session) -> None:
        atomic_write_json(self.snapshot_json, session.to_dict())

    def read_snapshot(self) -> dict | None:
        if not self.snapshot_json.exists():
            return None
        try:
            data = json.loads(self.snapshot_json.read_text(encoding="utf-8"))
            for key in ("session_id", "started_at", "duration", "on_time", "roster", "states"):
                data[key]
            return data
        except (ValueError, KeyError, TypeError):
            backup = self._backup_name(self.snapshot_json, "corrupt")
            shutil.move(self.snapshot_json, backup)
            self.issues.append(LoadIssue(self.snapshot_json.name, 0, 1, backup.name,
                                         "unreadable session snapshot was set aside"))
            return None

    def clear_snapshot(self) -> None:
        try:
            self.snapshot_json.unlink()
        except FileNotFoundError:
            pass

    # --- settings -----------------------------------------------------------
    def load_settings(self) -> dict:
        settings = dict(DEFAULT_SETTINGS)
        try:
            data = json.loads(self.settings_json.read_text(encoding="utf-8"))
            if isinstance(data, dict):
                settings.update({k: data[k] for k in DEFAULT_SETTINGS if k in data})
        except (FileNotFoundError, ValueError):
            pass
```

```
        return settings

    def save_settings(self, settings: dict) -> None:
        atomic_write_json(self.settings_json, settings)
```

12.5 Module 6 — analytics.py

```
"""
Module 6 - Attendance Analytics (75 % rule)
===========================================

For every student, over all saved sessions:

    attended = PRESENT + LATE + ON_DUTY
    counted  = sessions held - EXCUSED          (an excused class is not held against you)
    percent  = 100 * attended / counted

VIT requires 75 %. If a student is below it, the number of classes they must
attend in a row to get back to 75 % is the smallest x with

    (attended + x) / (counted + x) >= 0.75   =>   x = 3*counted - 4*attended
"""

from __future__ import annotations

from dataclasses import dataclass

from .fsm import ABSENT, EXCUSED, LATE, ON_DUTY, PRESENT

THRESHOLD = 75.0


@dataclass
class StudentSummary:
    regno: str
    name: str
    held: int = 0
    present: int = 0
    late: int = 0
    absent: int = 0
    excused: int = 0
    on_duty: int = 0

    @property
    def attended(self) -> int:
        return self.present + self.late + self.on_duty

    @property
    def counted(self) -> int:
        return self.held - self.excused

    @property
    def percentage(self) -> float | None:
        return None if self.counted <= 0 else 100.0 * self.attended / self.counted

    @property
    def classes_needed(self) -> int:
        return max(0, 3 * self.counted - 4 * self.attended)

    @property
    def status(self) -> str:
        pct = self.percentage
        if pct is None:
            return "No classes yet"
        return "OK" if pct >= THRESHOLD else "Below 75%"


def summarize(students: dict, attendance_rows: list[dict]) -> list[StudentSummary]:
    """students: {regno: Student}; attendance_rows: dicts from attendance.csv."""
    summary = {regno: StudentSummary(regno, s.name) for regno, s in students.items()}
    field = {PRESENT: "present", LATE: "late", ABSENT: "absent",
             EXCUSED: "excused", ON_DUTY: "on_duty"}
    for row in attendance_rows:
        item = summary.get(row.get("RegNo", ""))
        status = row.get("Status", "")
        if item is None or status not in field:
            continue
        item.held += 1
        setattr(item, field[status], getattr(item, field[status]) + 1)
    return list(summary.values())
```

12.6 Module 4 — GUI: key event handlers (attendance/gui/app.py)

```
    def _live_check(self, widget, live_label, path_label=None):
        text = widget.get().strip()
        mini = self.reg_mini if widget is self.reg_entry else None
        if not text:
            set_entry_state(widget, None)
            live_label.configure(text="DFA waiting in start state q0", fg=C["muted"])
            if path_label is not None:
                path_label.configure(text="")
            if mini is not None:
                mini.clear_run()
            return None
        tr = self.dfa.trace(text)
        if mini is not None:
            mini.show_run(tr, len(tr.steps), tr.accepted or tr.final_state == DEAD)
        if tr.accepted:
            kind, color = "ok", C["success"]
            msg = f"✓ Accepted — the run ends in final state {tr.final_state} ∈ F"
        elif tr.final_state == DEAD:
            kind, color = "error", C["danger"]
            pos = tr.error_position
            msg = (f"✕ Dead state at position {pos + 1} ('{text[pos]}') — state "
                   f"{tr.steps[-1].source} expects {describe(self.dfa.expected(tr.steps[-1].source))}")
        else:
            kind, color = "live", C["warning"]
            need = self.dfa.distance_to_accept(tr.final_state)
            msg = (f"… Live state {tr.final_state}: next expected "
                   f"{describe(self.dfa.expected(tr.final_state))} ({need} more symbol(s))")
        set_entry_state(widget, kind)
        live_label.configure(text=msg, fg=color)
        if path_label is not None:
            path_label.configure(text="Run: " + tr.path_text())
        return tr

    def _add_student(self):
        raw = self.reg_entry.get()
        self._live_check(self.reg_entry, self.reg_live, self.reg_path)
        try:
            student = self.store.add_student(raw, self.name_entry.get())
        except InvalidRegNo as e:
            self._recovered_from = None
            self._show_regno_error(self.reg_bar, raw, e.trace, e.suggestion, self._apply_reg_fix)
            self._live_check(self.reg_entry, self.reg_live, self.reg_path)
            set_entry_state(self.reg_entry, "error")
            offset = len(raw) - len(raw.lstrip())
            if e.trace.error_position is not None:
                p = e.trace.error_position + offset
                self.reg_entry.focus_set()
                self.reg_entry.selection_range(p, p + 1)
                self.reg_entry.icursor(p + 1)
            self._refresh_reports()
            return
        except DuplicateStudent as e:
            self.reg_bar.show(
                "warning", "Duplicate registration blocked",
                f"{e.regno} is already registered as {e.name} (added {e.added}). "
                f"No second record was created — the existing record is highlighted in the list.",
                actions=[("Clear form", self._clear_register)])
            self._select_student(e.regno)
            self._refresh_reports()
            return
        except InvalidName as e:
            self.reg_bar.show("error", "Invalid student name", str(e))
            set_entry_state(self.name_entry, "error")
            self.name_entry.focus_set()
            return
        set_entry_state(self.name_entry, None)
        if self._recovered_from:
            title = "Recovered input accepted and registered"
            msg = (f"'{self._recovered_from}' was corrected to {student.regno}. The DFA accepted it "
                   f"and {student.name} was saved to students.csv.")
        else:
            title = "Student registered"
            msg = f"{student.regno} · {student.name} was accepted by the DFA and saved to students.csv."
        trace = self.dfa.trace(student.regno)
        self.reg_bar.show("success", title, msg, details="Run: " + trace.path_text())
        self._recovered_from = None
        if self.session is not None and not self.session.closed:
            self.session.add_student(student.regno, student.name)
```

```
            self.store.write_snapshot(self.session)
        self.reg_entry.delete(0, "end")
        self.name_entry.delete(0, "end")
        self._live_check(self.reg_entry, self.reg_live, self.reg_path)
        self._refresh_all()
        self._select_student(student.regno)
        self.reg_entry.focus_set()

    def _end_session(self, auto=False, restored=False):
        s = self.session
        if s is None or s.closed:
            self.att_bar.show("info", "No running session", "Start a session first.")
            return
        records = s.close()
        for regno, rec in records:
            self.store.log_transition(s.session_id, regno, rec)
        self.store.save_session(s)
        self.store.clear_snapshot()
        if restored:
            title = "Session restored and closed — its time had already run out"
        elif auto:
            title = "Time limit reached — session closed automatically"
        else:
            title = "Session ended by faculty"
        msg = (f"{len(records)} unmarked student(s) received the TIMEOUT event "
               f"(NOT_MARKED → ABSENT). Final states of all {len(s.machines)} students were saved "
               f"to attendance.csv.")
        details = "\n".join(f"{r}  {s.roster.get(r, ''):<22} NOT_MARKED --TIMEOUT--> ABSENT"
                            for r, _ in records) or None
        self.att_bar.show("warning" if records else "success", title, msg, details=details)
        self._refresh_all()
        self._update_timer()

    def _mark(self):
        raw = self.mark_entry.get().strip()
        s = self.session
        self._live_check(self.mark_entry, self.mark_live)
        if not raw:
            self.att_bar.show("info", "Enter a registration number",
                              "Type or scan the student's registration number and press Enter.")
            return
        if s is None or s.closed:
            self.store.log_reject("SESSION", raw, "mark attempted with no active session")
            self.att_bar.show("error", "No active attendance session",
                              f"{raw} cannot be marked because no session is running. Attendance "
                              f"is only accepted inside a timed session window.",
                              actions=[("Start session now", self._start_and_mark)])
            self._refresh_reports()
            return
        trace = self.dfa.trace(raw)
        if not trace.accepted:
            suggestion = self.dfa.suggest(raw)
            self.store.log_reject("MARK", raw, trace.reason)
            self._show_regno_error(self.att_bar, raw, trace, suggestion, self._apply_mark_fix)
            self._live_check(self.mark_entry, self.mark_live)
            self._refresh_reports()
            return
        if raw not in self.store.students:
            self.store.log_reject("MARK", raw, "registration number not registered")
            self.att_bar.show("error", "Student not registered",
                              f"{raw} is a valid registration number (accepted by the DFA) but it "
                              f"is not in students.csv, so it cannot be marked.",
                              actions=[("Register this student", lambda: self._register_from_mark(raw))])
            self._refresh_reports()
            return
        if raw not in s.machines:
            s.add_student(raw, self.store.students[raw].name)
        try:
            rec = s.mark(raw)
        except InvalidTransition as e:
            self.store.log_reject("FSM", raw, str(e))
            fsm = s.machines[raw]
            history = "\n".join(str(h) for h in fsm.history)
            self.att_bar.show("warning", "Duplicate marking blocked by the FSM",
                              f"{raw} ({s.roster.get(raw)}) is already {fsm.state}"
                              f"{' since ' + s.marked_at[raw] if raw in s.marked_at else ''}. "
                              f"{e} The attempt was logged in reject_log.csv.",
                              details="History of this student:\n" + history)
            self._select_att(raw)
            self._refresh_reports()
            return
```

```
        except SessionClosed as e:
            self.att_bar.show("error", "Session closed", str(e))
            return
        self.store.log_transition(s.session_id, raw, rec)
        self.store.write_snapshot(s)
        on_time = rec.target != "LATE"
        self.att_bar.show("success" if on_time else "warning",
                          f"{raw} marked {rec.target}" + (" (on time)" if on_time else " (after the cut-off)"),
                          f"{s.roster.get(raw)}:  {rec.source} --{rec.event}--> {rec.target}  at {rec.time}")
        self.mark_entry.delete(0, "end")
        self._live_check(self.mark_entry, self.mark_live)
        self._refresh_attendance()
        self._select_att(raw)

    def _startup_checks(self):
        snapshot = self.store.read_snapshot()
        if self.store.issues:
            lines = []
            for i in self.store.issues:
                extra = (f" — {i.good_rows} good row(s) kept, {i.bad_rows} moved"
                         if i.bad_rows else "")
                lines.append(f"• {i.file}: {i.note}{extra}\n    backup → {i.backup}")
            self.reg_bar.show("warning", "Recovered from damaged data files",
                              "Problems were found while loading the data files and were repaired "
                              "automatically. Nothing was deleted: the damaged content was copied "
                              "to the backup file(s) below.", details="\n".join(lines))
        if snapshot:
            try:
                session = AttendanceSession.from_dict(snapshot, clock=self.clock)
            except (KeyError, ValueError, TypeError) as e:
                self.att_bar.show("warning", "Saved session could not be restored",
                                  f"The crash-recovery snapshot is damaged ({e}). It was ignored.")
                self.store.clear_snapshot()
                return
            self._offer_restore(session)

    def _restore_session(self, s):
        self.session = s
        marked = sum(1 for m in s.machines.values() if m.is_marked)
        if s.expired:
            self._end_session(auto=True, restored=True)
            return
        self.store.write_snapshot(s)
        mm, ss = divmod(int(s.remaining()), 60)
        self.att_bar.show("success", "Session restored — no marks were lost",
                          f"{marked} mark(s) recovered from active_session.json. The timer "
                          f"continues with {mm:02d}:{ss:02d} left ({PHASE_STYLE[s.phase()][0].lower()}).")
        self._refresh_all()
        self._update_timer()
```

12.7 main.py

```
"""Start the Student Attendance System.

    python main.py                  # data is kept in ./data
    python main.py --data-dir demo  # use another folder
"""

from attendance.gui.app import run

if __name__ == "__main__":
    run()
```

13. Screenshots of the Error Segments

Every error is detected before it can damage the data, explained in plain words, and logged. The screenshots were captured from the running application.

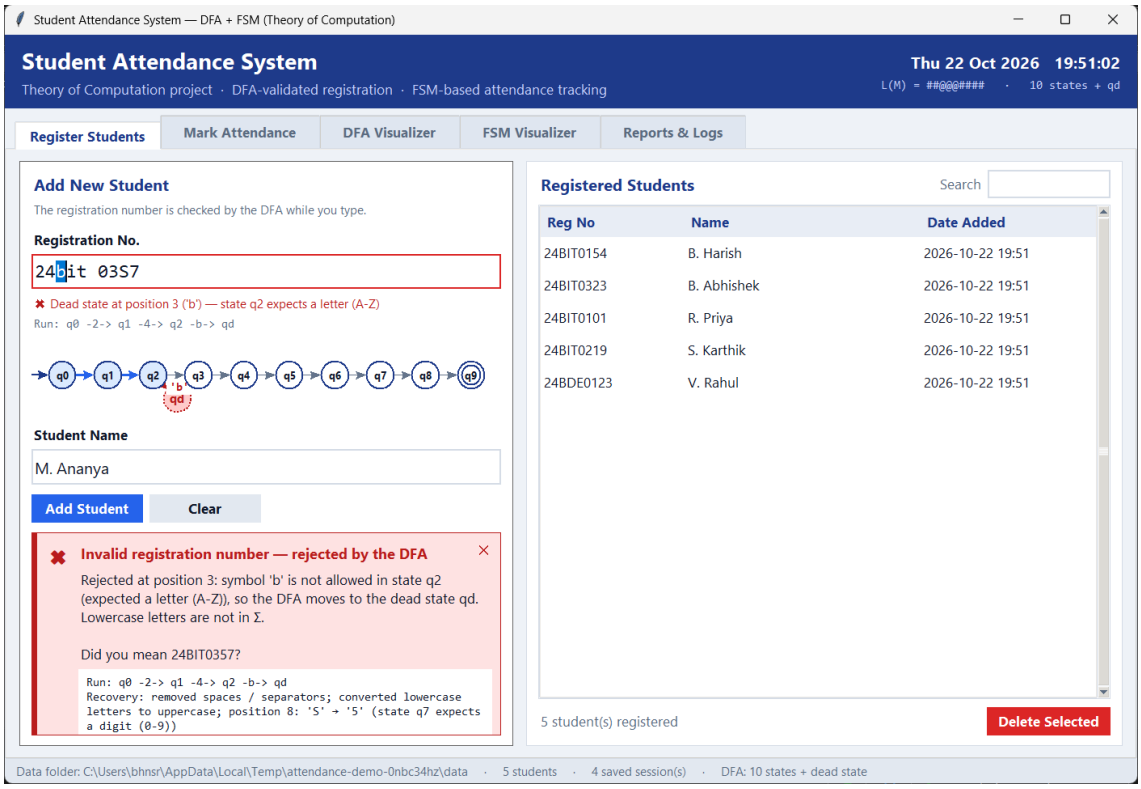


Figure 8: E1 — Invalid registration number. *`24bit 03S7` is rejected: the DFA reaches the dead state qd at position 3 ('b' is lowercase and not in Σ). The banner shows the state, the expected symbol class and the DFA path, and the mini diagram shows q2 → qd.*

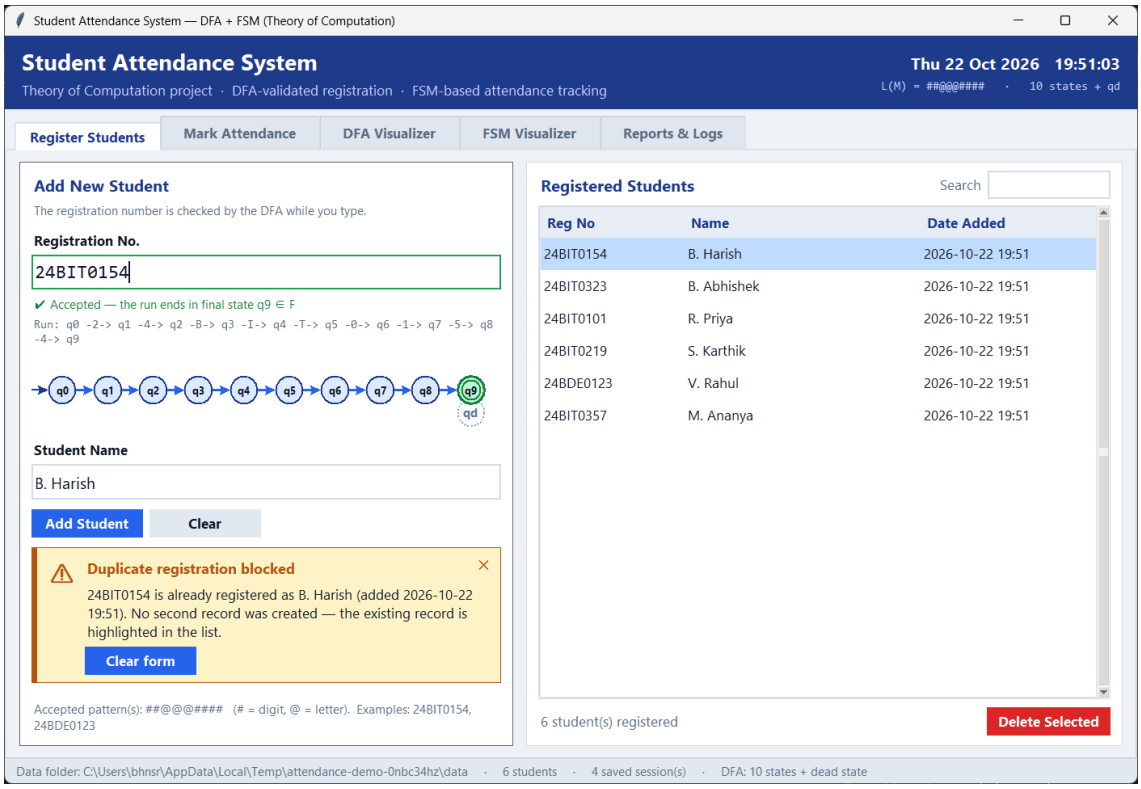


Figure 9: E2 — Duplicate registration. *24BIT0154 already exists. No second record is created.*

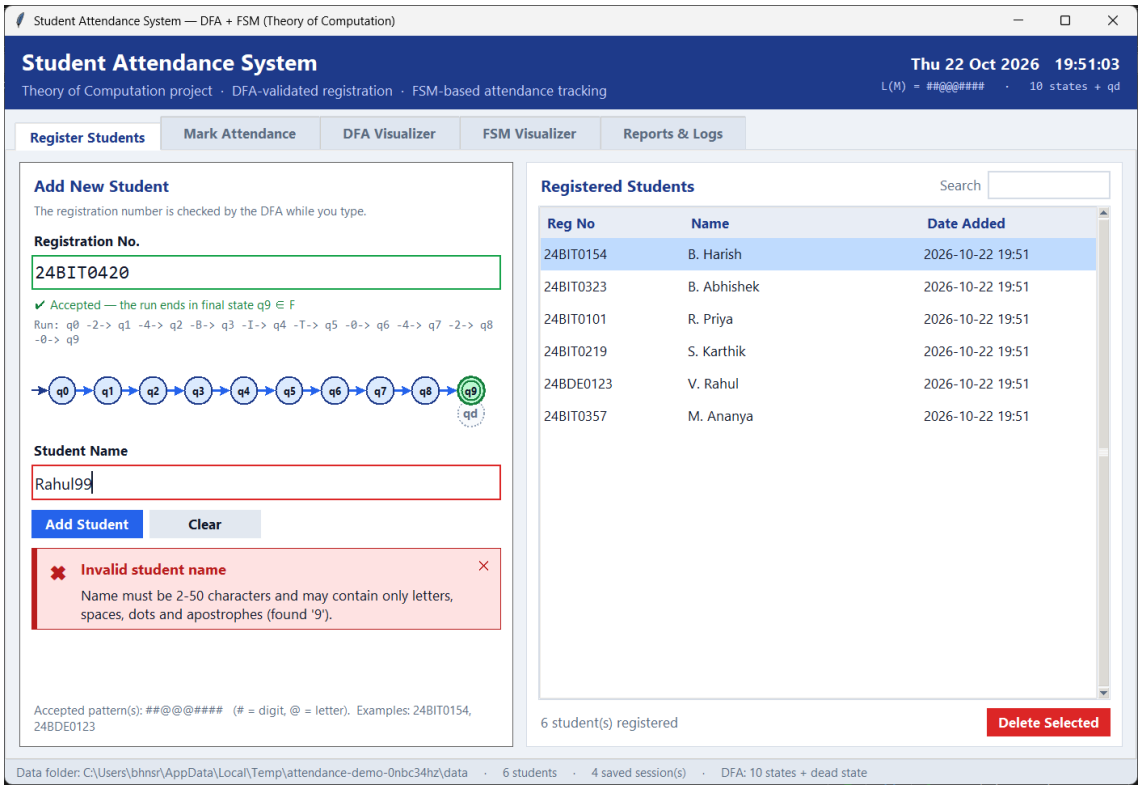


Figure 10: E3 — Invalid student name. *Rahul99 contains a digit, so it is rejected and the name field is marked.*

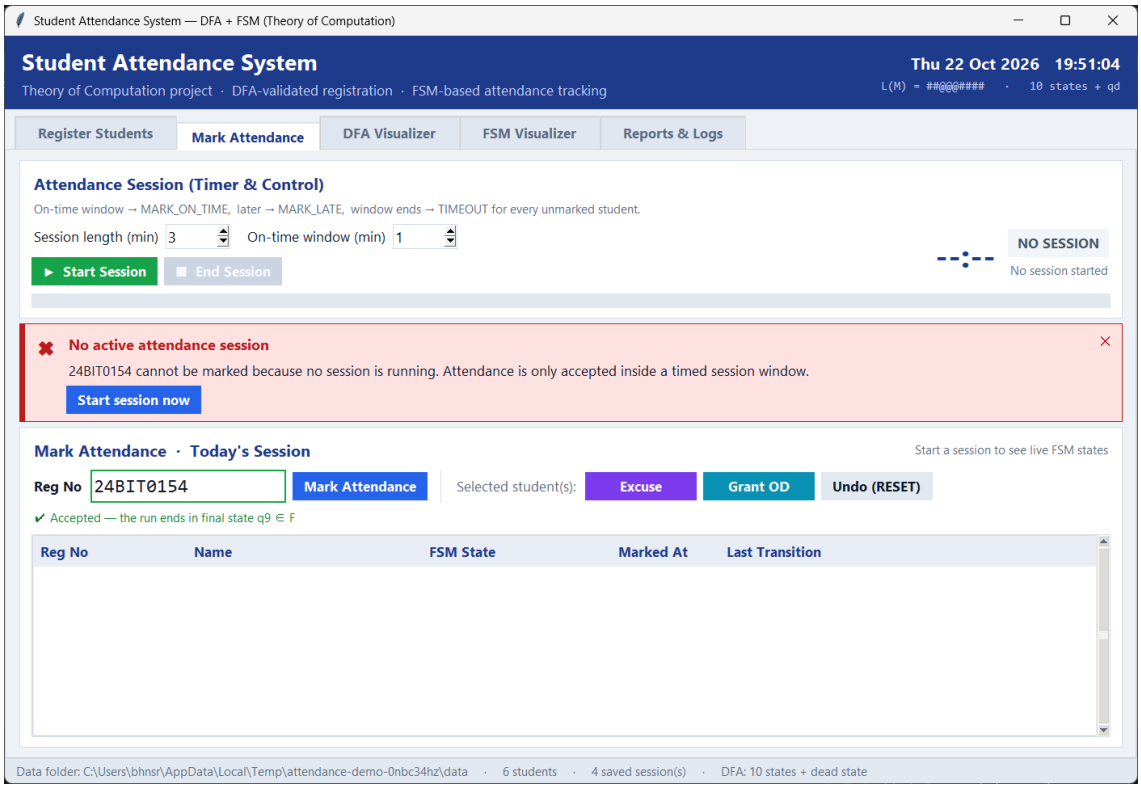


Figure 11: E4 — Marking without an active session. *Attendance is only accepted inside a timed session window.*

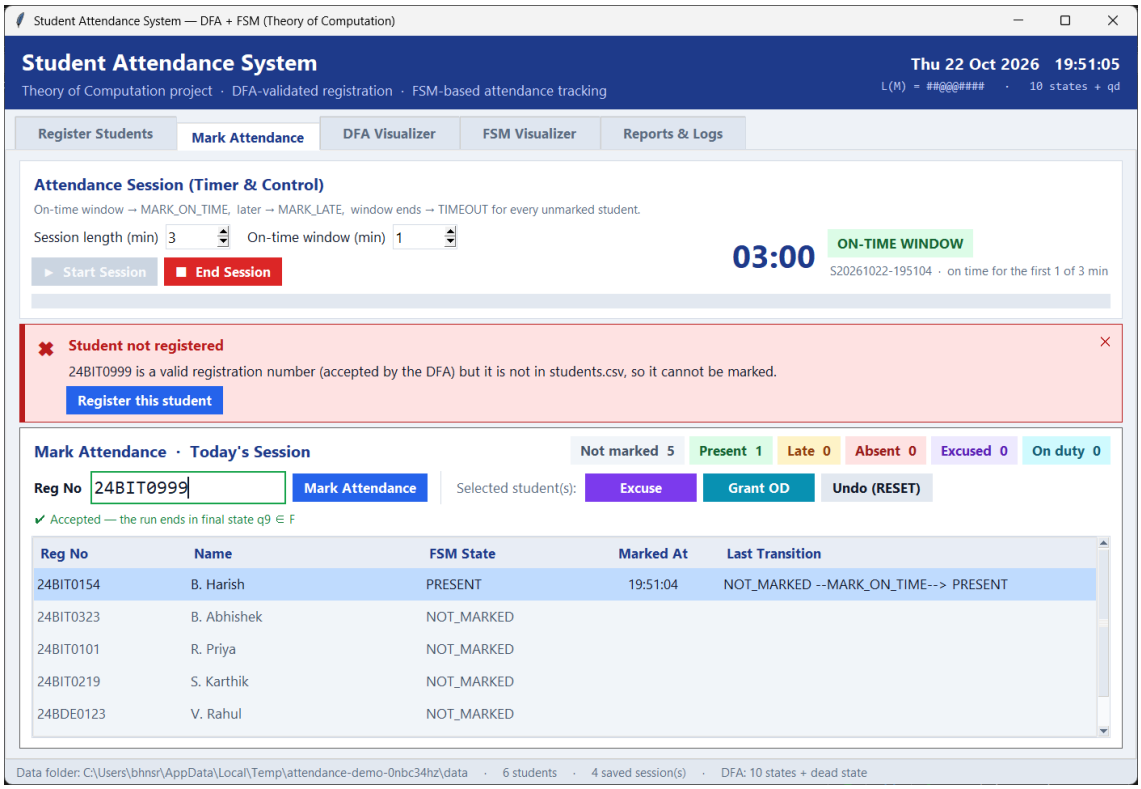


Figure 12: E5 — Unregistered student. *24BIT0999 is accepted by the DFA but is not in students.csv.*

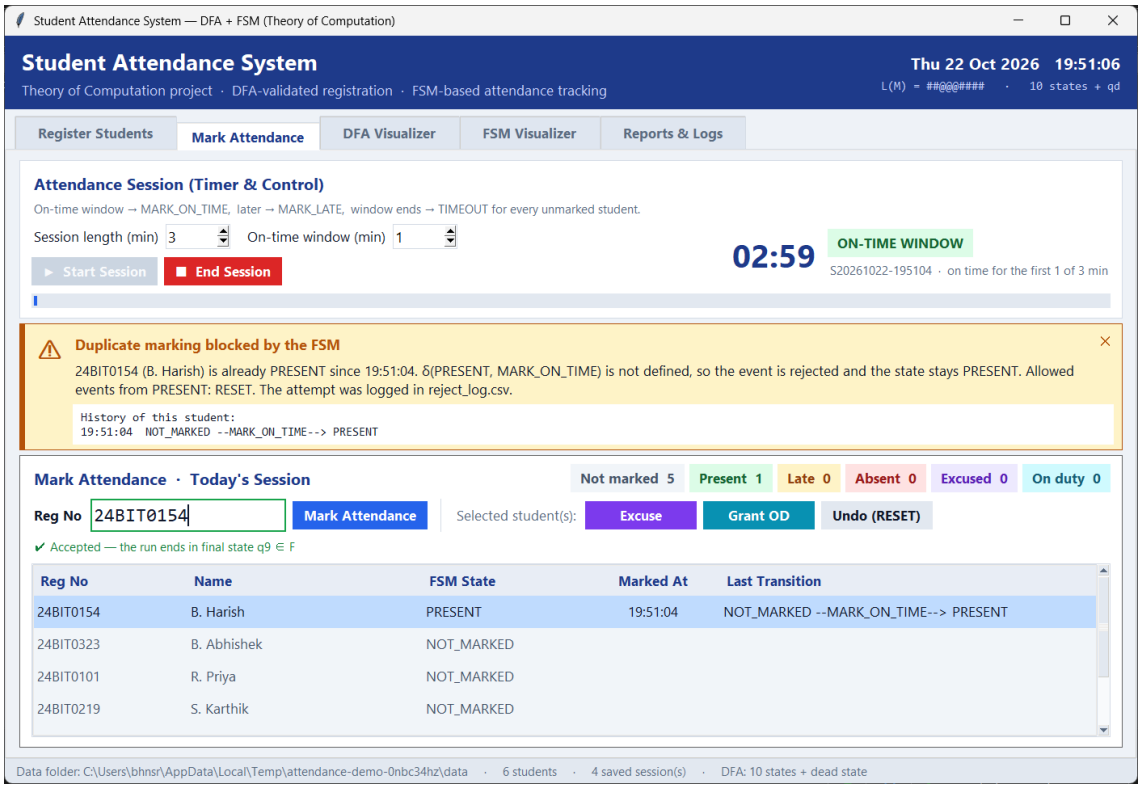


Figure 13: E6 — Double / proxy marking. *δ(PRESENT, MARK_ON_TIME) is undefined, so the FSM rejects the event, keeps the state PRESENT, and logs the attempt.*

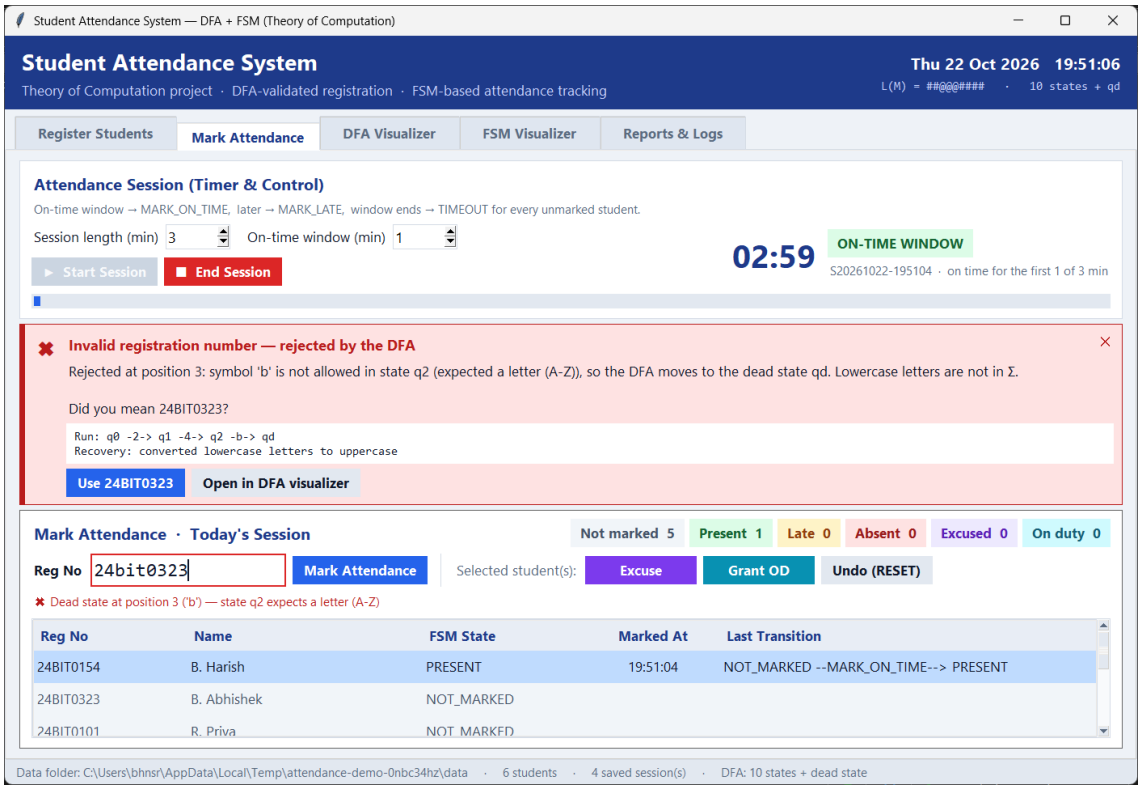


Figure 14: E7 — Typo while marking. *`24bit0323` is rejected by the DFA.*

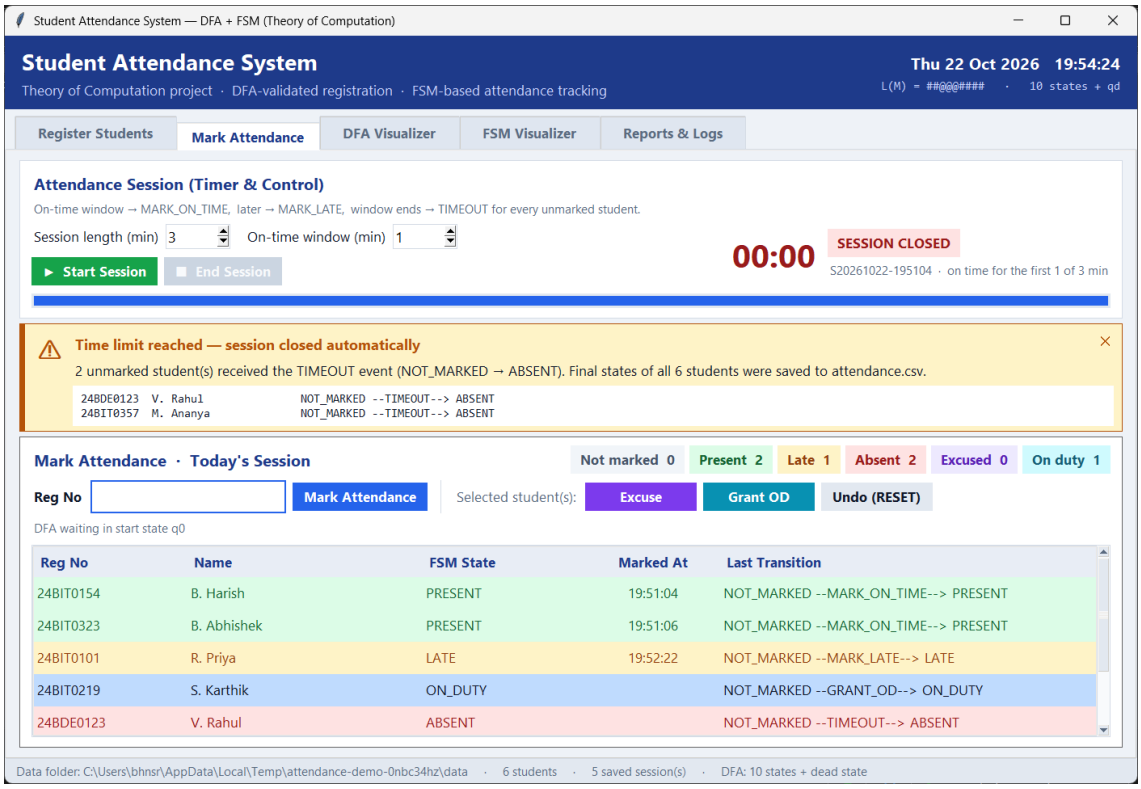


Figure 15: E8 — Time-out (error mode). *The attendance window expired, so TIMEOUT moved the two unmarked students NOT_MARKED → ABSENT automatically.*

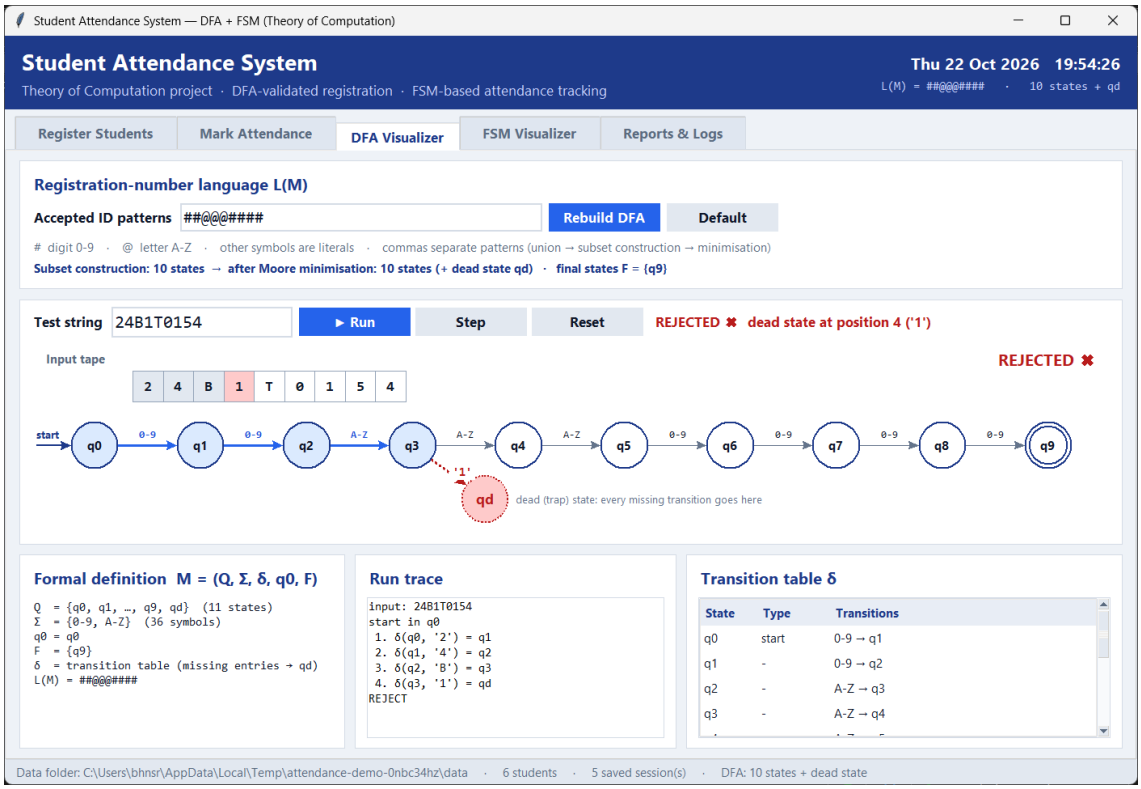


Figure 16: E9 — Rejection shown in the DFA visualizer. *For 24B1T0154 the symbol '1' in state q3 leads to the dead state qd.*

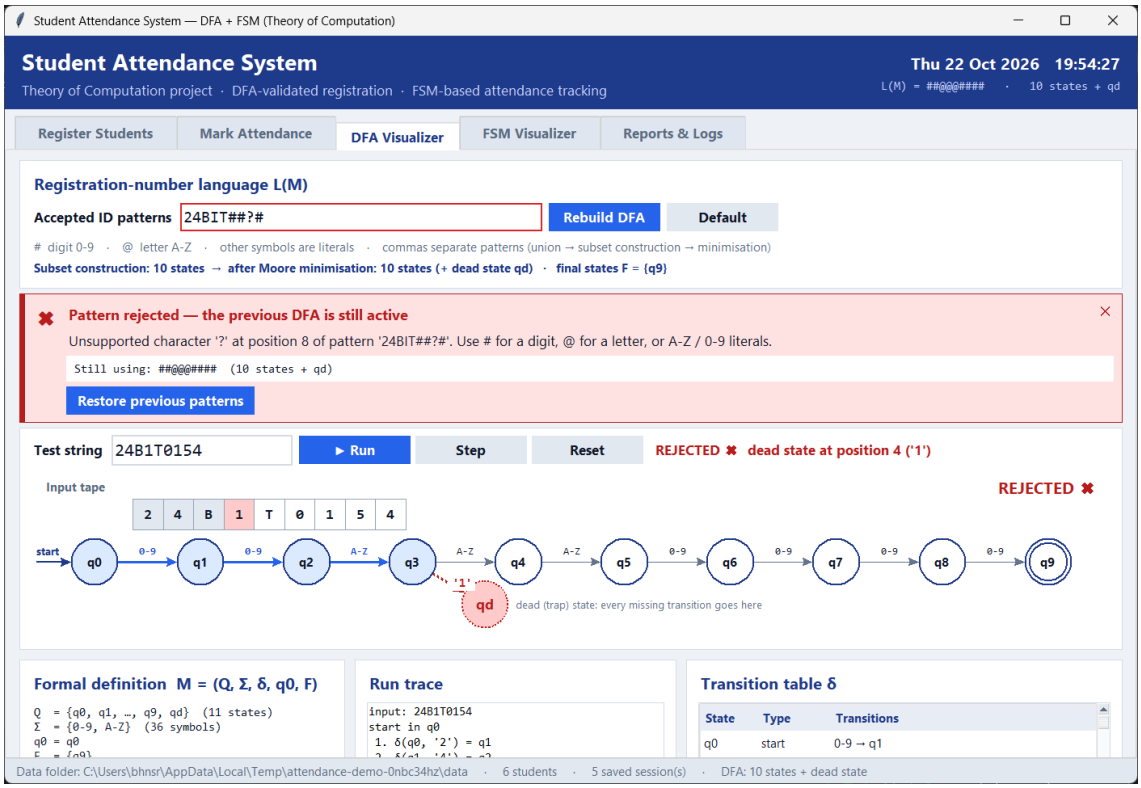


Figure 17: E10 — Invalid ID pattern. *24BIT##?# contains '?', so it cannot be compiled into a DFA.*

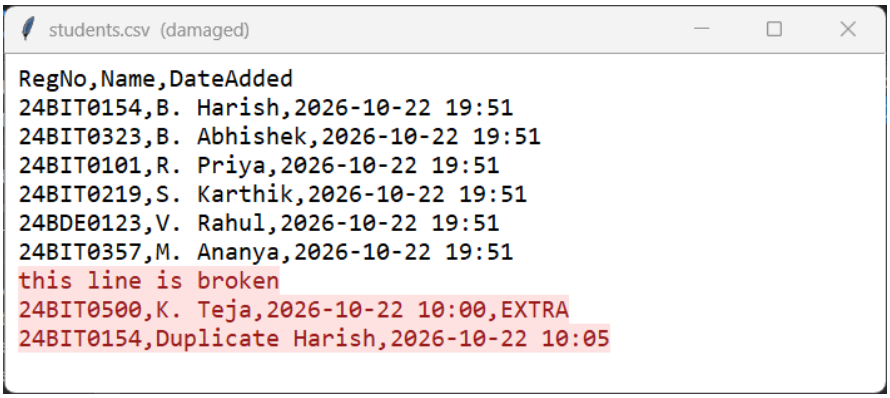


Figure 18: E11 — Corrupted data file. *students.csv contains a broken line, a row with an extra field and a duplicate ID (highlighted).*

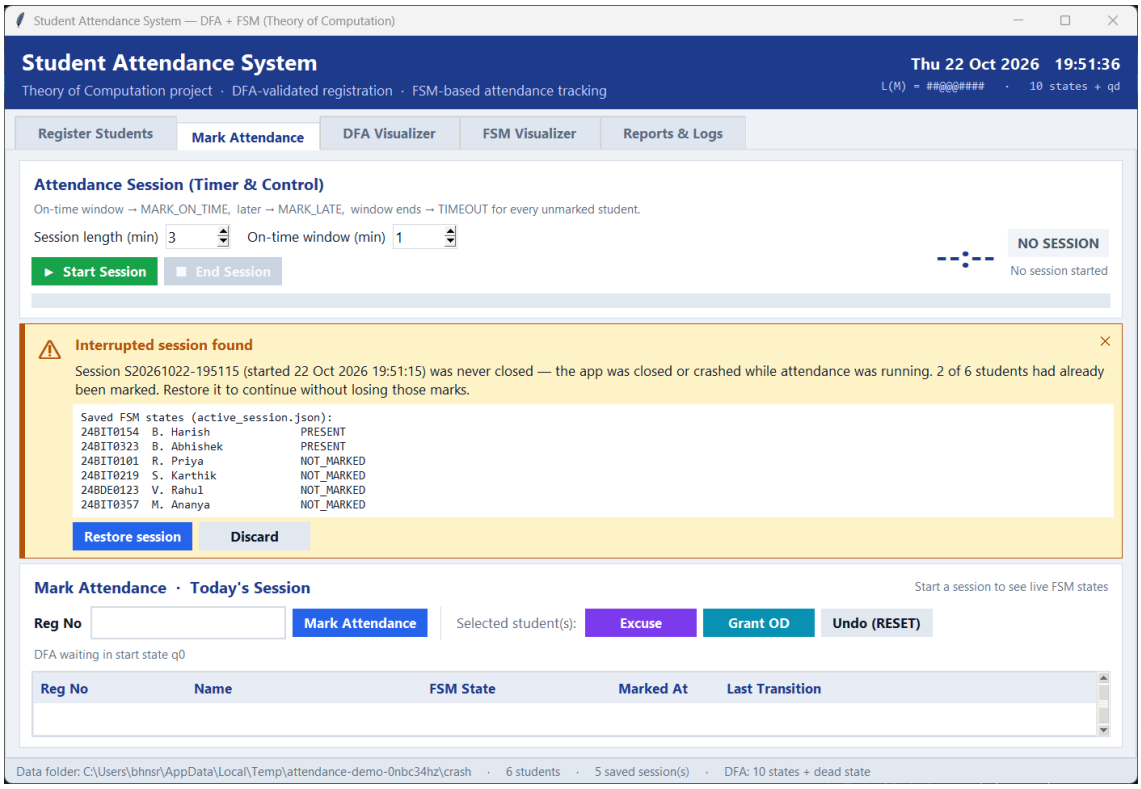


Figure 19: E12 — Application crash during a session. *The app was closed while a session was running. On restart the unfinished session is detected.*

14. Screenshots of the Error Recovered Segments

For every error the system offers a way back to a correct state without losing data.

Error	Recovery	Figure
E1 Invalid reg. no.	DFA-guided correction “Use 24BIT0357”, then accepted in q9 and saved	R1
E2 Duplicate	Existing record highlighted; nothing written (visible in E2)	E2
E4 No session	“Start session now” starts the session and marks the student	R2
E5 Unregistered	“Register this student” opens the Register tab with the ID filled in	—
E6 Double marking	State unchanged; attempt recorded in reject_log.csv (Reports tab)	R6
E7 Typo while marking	“Use 24BIT0323” corrects and marks in one click	R3
E8 Time-out	Admin override: ABSENT --EXCUSE--> EXCUSED after the session	R4
E10 Bad pattern	Previous DFA kept active; corrected pattern list rebuilt	R5
E11 Corrupted CSV	Bad rows quarantined to a backup file, good rows loaded	R7
E12 Crash	Session restored from active_session.json, no marks lost	R8

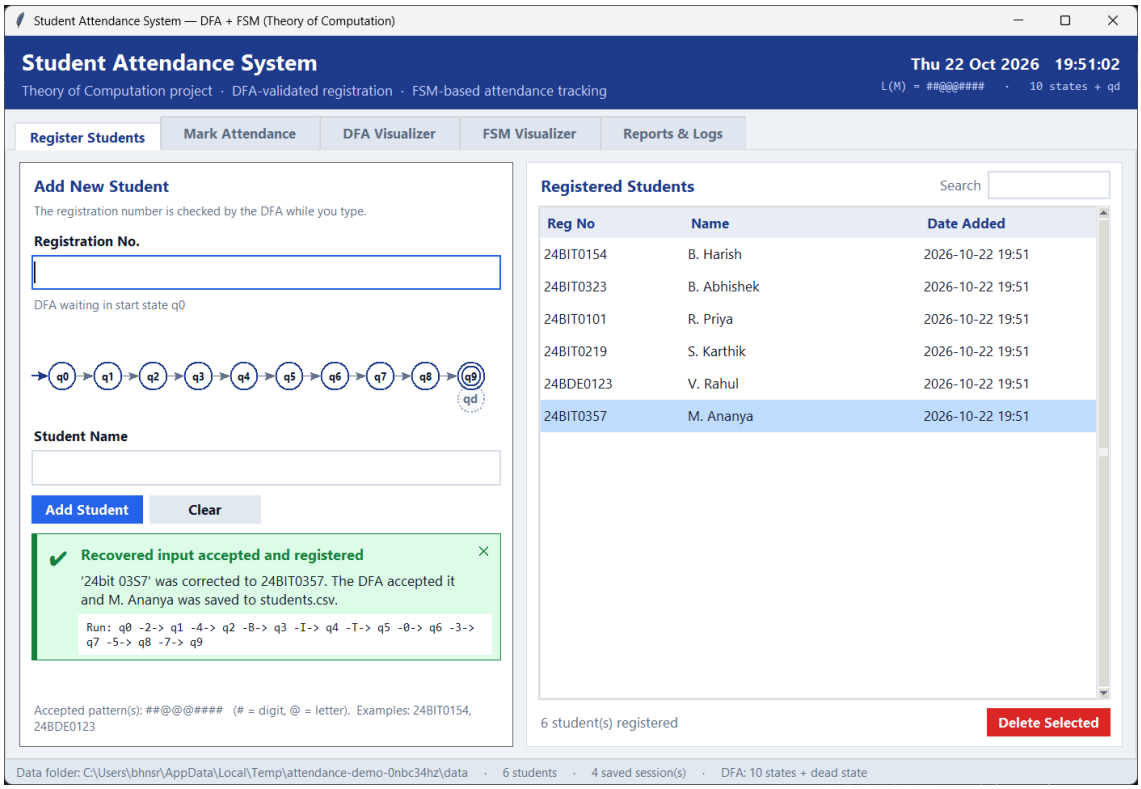


Figure 20: R1 — Recovered registration. *'24bit 03S7' was corrected to 24BIT0357. The DFA accepted it (q0 → … → q9) and M. Ananya was registered.*

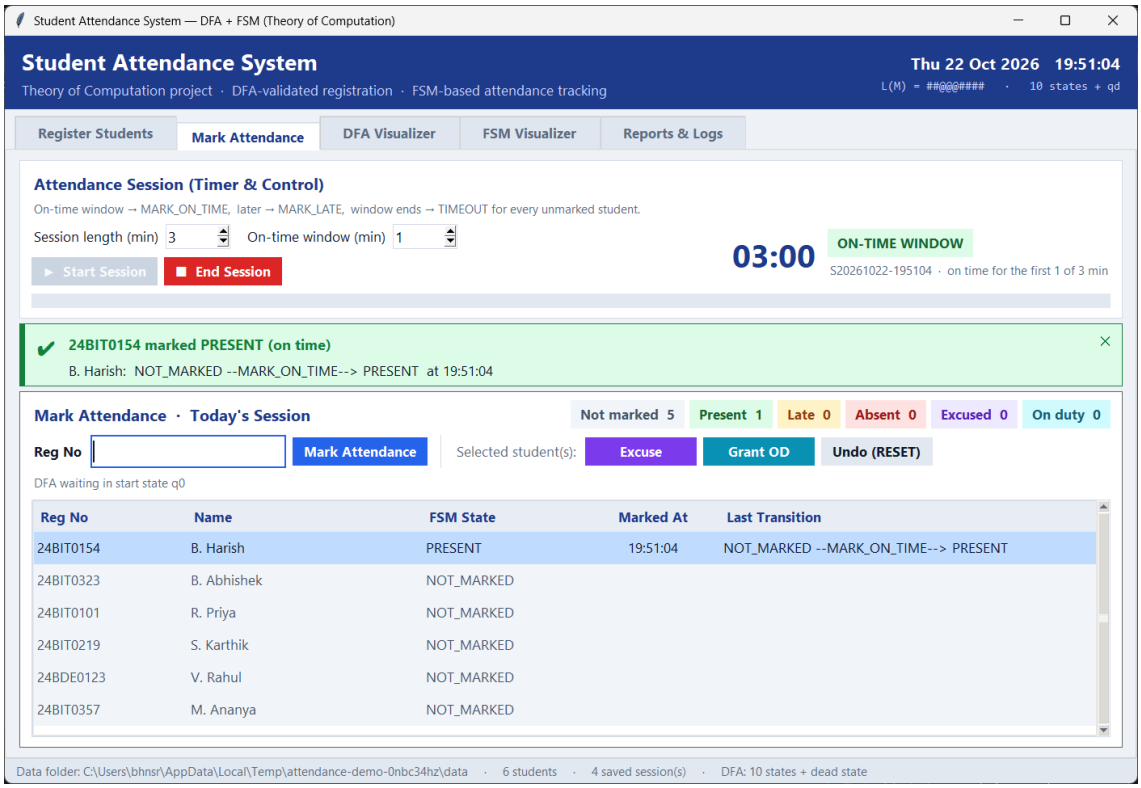


Figure 21: R2 — Recovered from "no session". *Start session now started the timer and marked 24BIT0154 PRESENT (NOT_MARKED --MARK_ON_TIME--> PRESENT).*

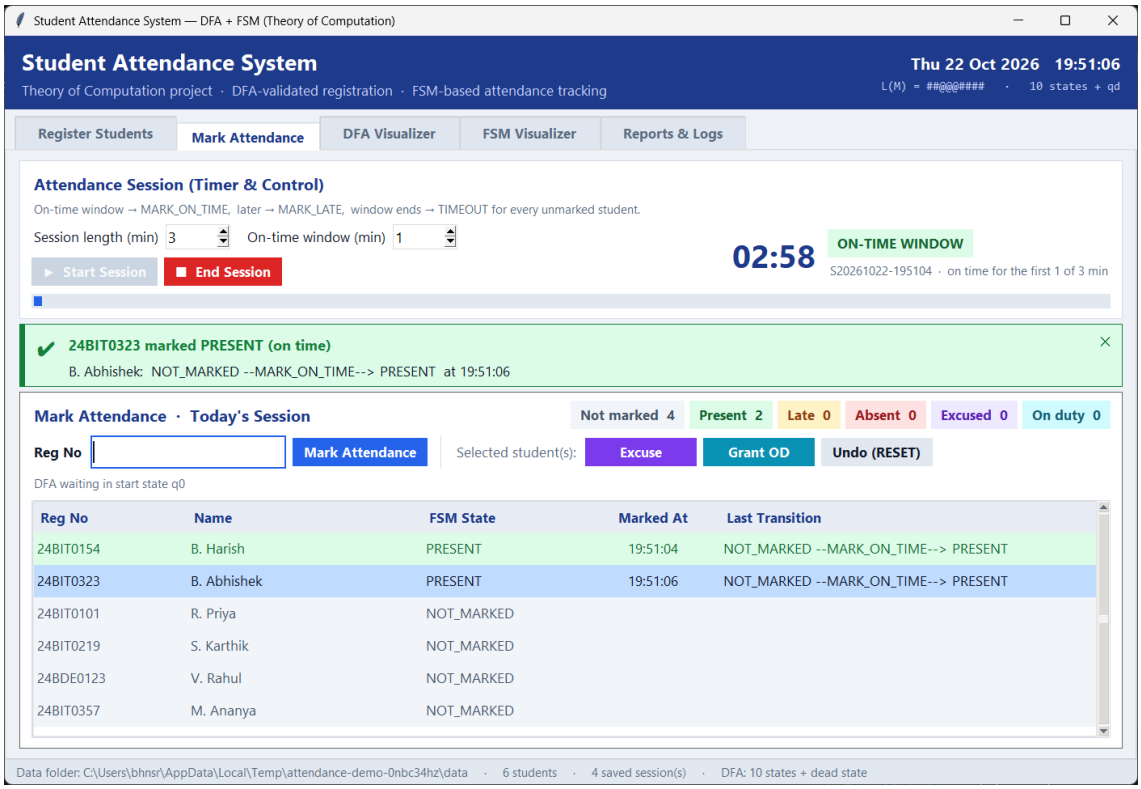


Figure 22: R3 — Recovered typo while marking. *The suggestion 24BIT0323 was applied and B. Abhishek was marked PRESENT.*

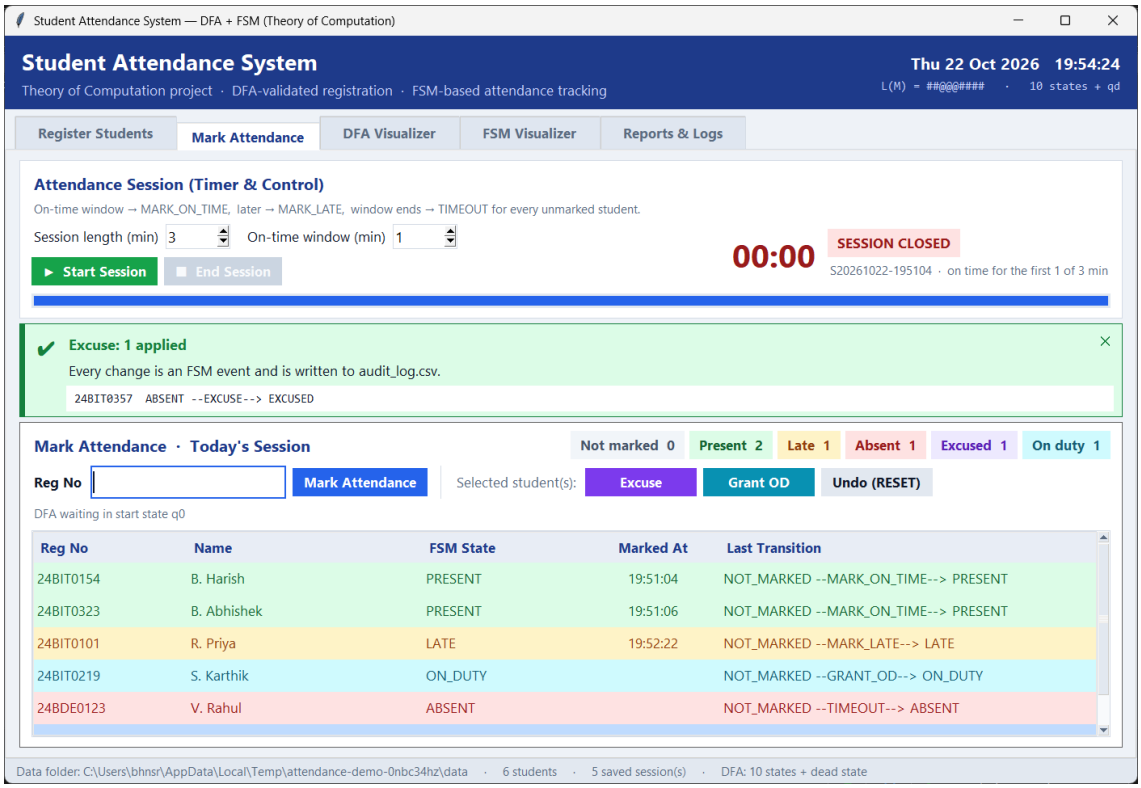


Figure 23: R4 — Recovery after time-out. *A valid absence is corrected by the admin override ABSENT --EXCUSE--> EXCUSED, and the session is re-saved.*

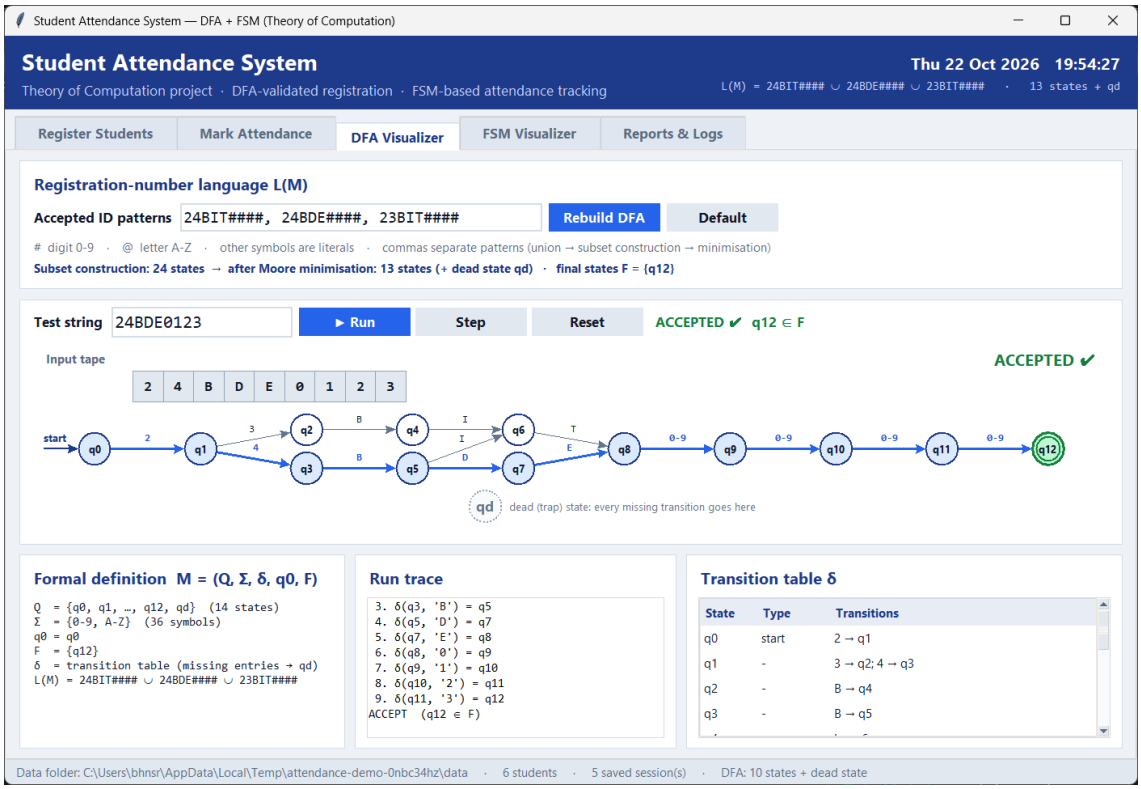


Figure 24: R5 — Recovered pattern. *After the invalid pattern was rejected (the old DFA stayed active), a corrected pattern list was compiled: subset construction gives 24 states, minimised to 13.*

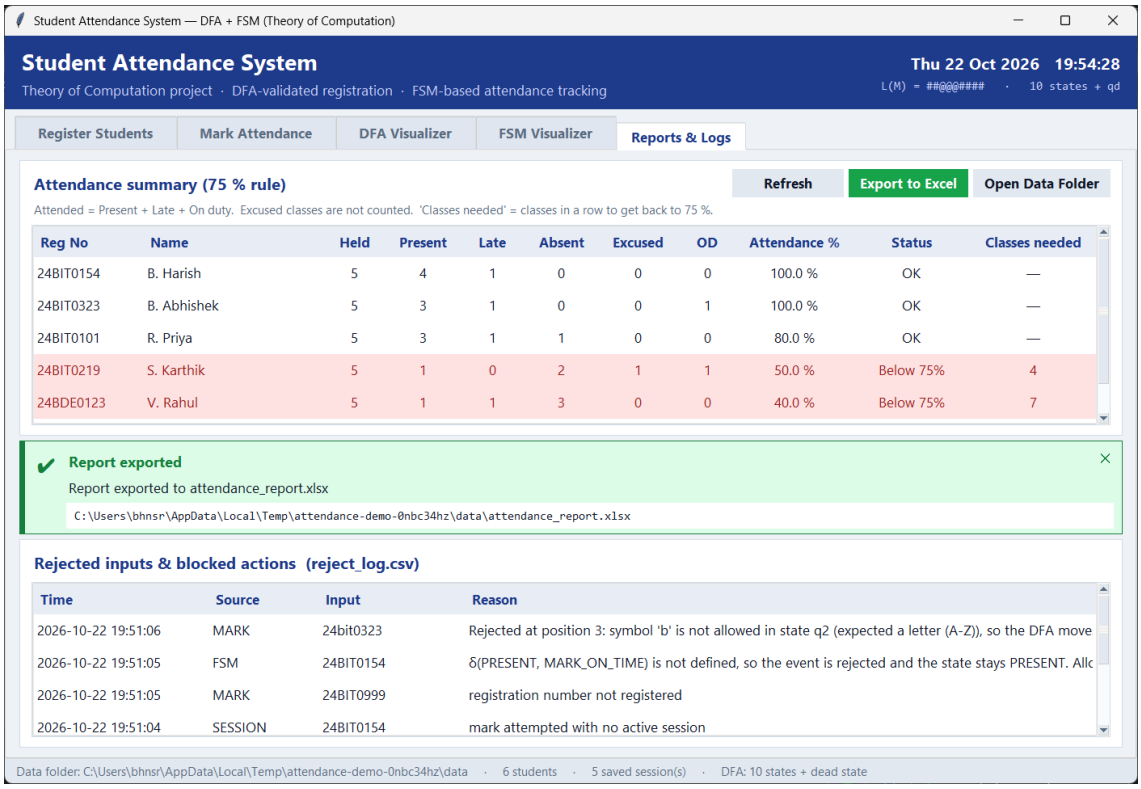


Figure 25: R6 — Blocked attempts are recorded. *The double mark, the typo and the unregistered ID appear in the reject log, and the attendance data stays correct.*

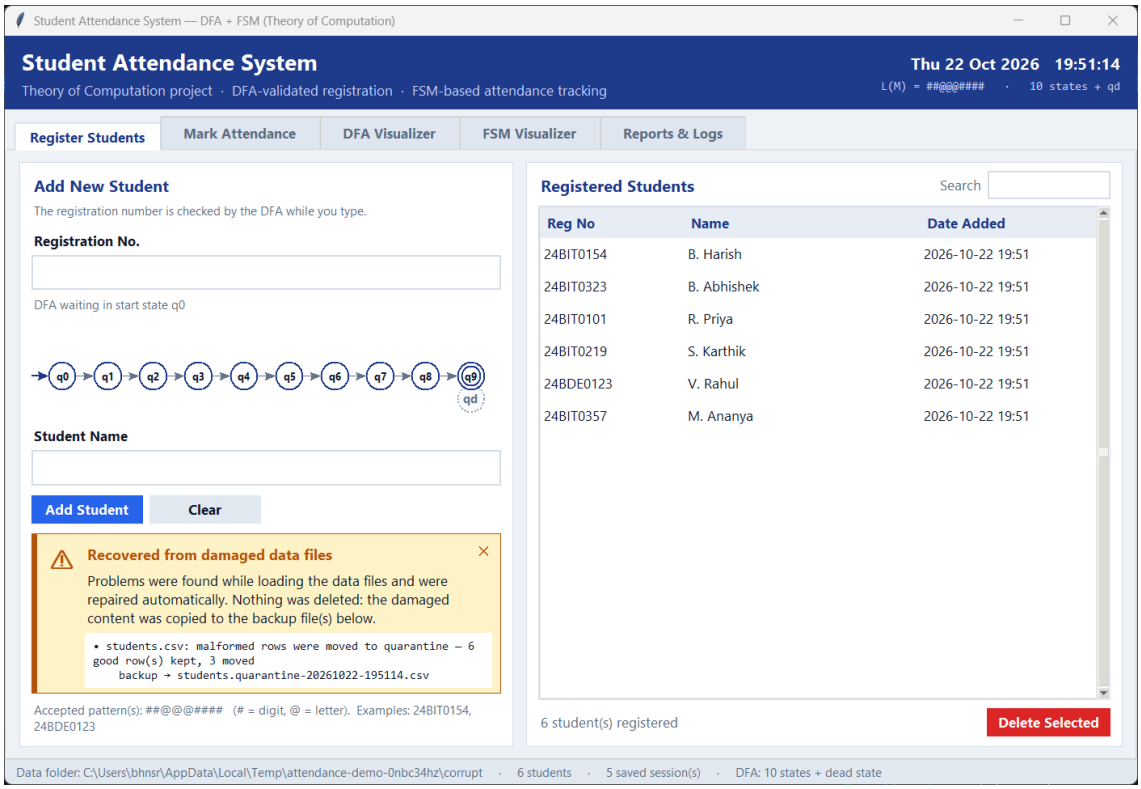


Figure 26: R7 — Recovered corrupted data. *Three damaged rows were moved to students.quarantine-<time>.csv, the 6 good rows loaded, and nothing was lost.*

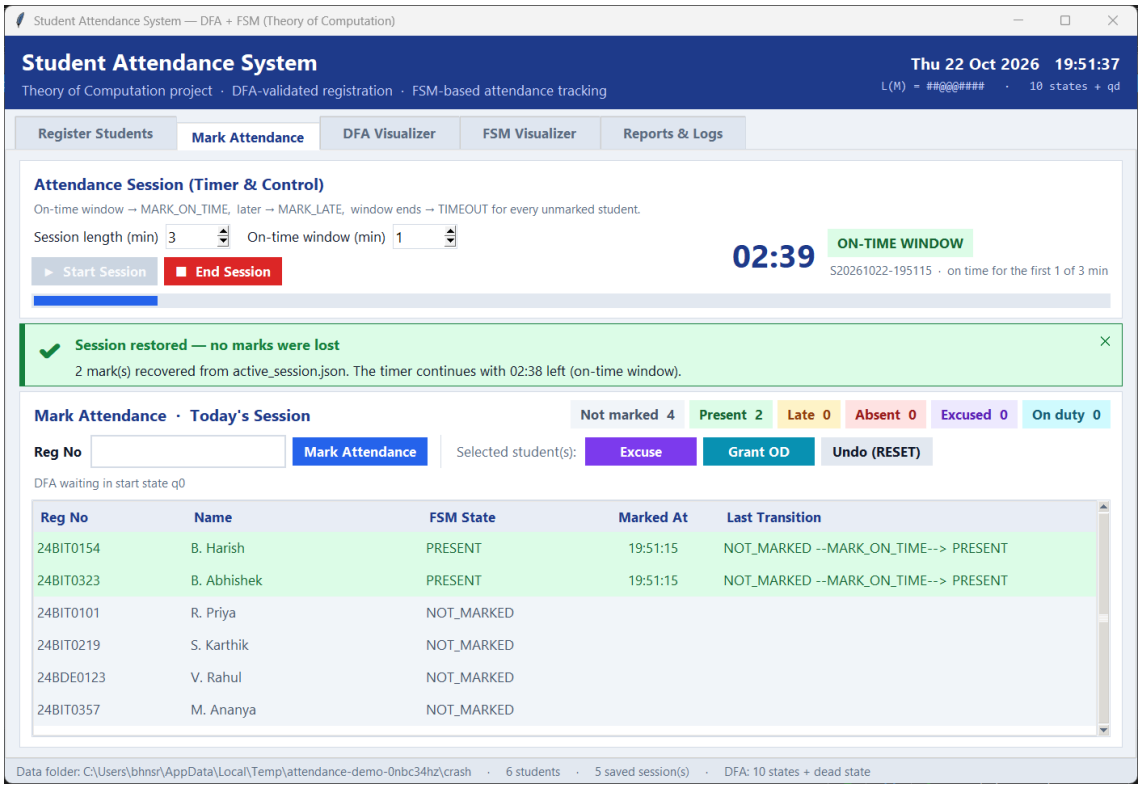


Figure 27: R8 — Recovered crashed session. *Restore session brought back both marks and the running timer (no marks lost).*

15. Possible Future Work

- **Richer patterns:** accept full regular expressions (with *, +, |) using Thompson's construction (regex → NFA) before the subset construction.
- **Faster minimisation:** Hopcroft's O(n log n) algorithm for very large pattern sets.
- **Database:** move from CSV to SQLite or MySQL for many classes and concurrent access.
- **Authentication and roles:** faculty login before marking or overriding attendance.
- **Contact-less marking:** QR code, RFID or face recognition as new input events for the same FSM.
- **Web / mobile version:** a Flask or Django backend with a React or Android front end.
- **Notifications:** automatic e-mail or SMS to students who fall below 75 %.
- **Timetable integration:** start sessions automatically from the class timetable.

16. Conclusion

The project shows that ideas from Theory of Computation solve real problems in a classroom tool. Treating valid registration numbers as a regular language and recognising it with a DFA gives validation that is fast (O(n)), exact and explainable. The same automaton also drives live feedback, the visualizer and safe automatic correction. Generating the DFA from patterns, with the subset construction and minimisation, makes the system flexible without any code changes.

Modelling attendance as a deterministic, timer-driven FSM makes the rules explicit. Every status change is a defined transition, illegal events such as double marking are rejected, and the time-out is applied fairly to everyone. With crash recovery, quarantine of corrupted data, atomic writes, analytics and 67 automated tests, the result is a complete and reliable application.

17. References

Base papers and base links are highlighted.

[1] **M. O. Rabin and D. Scott, "Finite Automata and Their Decision Problems," IBM Journal of Research and Development, vol. 3, no. 2, pp. 114–125, 1959.** *(Base paper: DFA, NFA and the subset construction used in Module 1.)*

[2] **E. F. Moore, "Gedanken-Experiments on Sequential Machines," in Automata Studies, C. E. Shannon and J. McCarthy, Eds. Princeton University Press, 1956, pp. 129–153.** *(Base paper: sequential (finite state) machines and state equivalence, used for the FSM and for DFA minimisation.)*

[3] G. H. Mealy, "A Method for Synthesizing Sequential Circuits," Bell System Technical Journal, vol. 34, no. 5, pp. 1045–1079, 1955.

[4] J. E. Hopcroft, "An n log n Algorithm for Minimizing States in a Finite Automaton," in Theory of Machines and Computations, Z. Kohavi and A. Paz, Eds. Academic Press, 1971, pp. 189–196.

[5] K. Thompson, "Programming Techniques: Regular Expression Search Algorithm," Communications of the ACM, vol. 11, no. 6, pp. 419–422, 1968.

[6] J. E. Hopcroft, R. Motwani and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed. Pearson, 2006.

[7] M. Sipser, Introduction to the Theory of Computation, 3rd ed. Cengage Learning, 2013.

[8] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson, 2006 (error recovery strategies).

Base links

[9] **Project repository (this work): https://github.com/HARISH0789/TOC-Student-Attendance-System**

[10] **Reference implementation of a DFA/FSM attendance system (base link): https://github.com/hanumasathvik75/STUDENT-ATTENDENCE-SYSTEM**

[11] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," https://docs.python.org/3/library/tkinter.html

[12] Python Software Foundation, "csv — CSV File Reading and Writing," https://docs.python.org/3/library/csv.html

[13] openpyxl documentation, https://openpyxl.readthedocs.io/